

DRC Results Generator

Moses Mburu

Table of contents

1 Purpose	3
2 Run Configuration	3
3 Build Descriptive Objects	7
4 Build Classical Decomposition Objects	9
5 Fit Concentration-Index Trees	22
6 Fit Lecturer Rpart Comparison Trees	27
6.0.1 Lecturer rpart method 1: absolute CI impurity	29
6.0.2 Lecturer rpart method 2: robust absolute CI impurity	29
7 Forest and SHAP Objects	29
8 Compare Methods	51
9 Save Results Objects	53

```
knitr::opts_chunk$set(  
  fig.width = 9,  
  fig.height = 5.5,  
  dpi = 500,  
  dev = "png"  
)  
  
library(data.table)  
library(ggplot2)  
library(grid)  
library(knitr)  
library(ineqTrees)  
library(partykit)  
library(here)  
source(here("R", "report_helpers.R"))  
source(here("R", "ci_rpart_comparison_methods.R"))  
  
kableExtra::use_latex_packages()
```

```

env_int <- function(name, default) {
  value <- suppressWarnings(as.integer(Sys.getenv(name, unset = NA_character_)))
  as.integer(ifelse(is.na(value) || value <= 0L, default, value))
}

env_num_or_null <- function(name, default = NULL) {
  value <- Sys.getenv(name, unset = NA_character_)
  if (is.na(value) || !nzchar(value)) {
    return(default)
  }

  parsed <- suppressWarnings(as.numeric(value))
  if (is.na(parsed)) {
    return(default)
  }
  parsed
}

```

```

default_params <- list(
  analysis_rows = 1L,
  tuning_folds = 10L,
  undersample = 0.4
)

runtime_params <- utils::modifyList(default_params, as.list(params))

runtime_params$analysis_rows <- env_int("ANALYSIS_ROWS", runtime_params$analysis_rows)
runtime_params$tuning_folds <- max(2L, env_int("TUNING_FOLDS", runtime_params$tuning_folds))
runtime_params$undersample <- env_num_or_null("UNDERSAMPLE", runtime_params$undersample)

shap_n <- 300L
shap_nsim <- 8L

```

```

criterion_types <- c("CI", "CIg", "CIc", "L")
rineq_ci_types <- c("CI", "CIg", "CIc", "CIw")
results_object_file <- here("data", "drc_report_results_objects.rda")
source_data_file <- here("prev_analysis", "DR Congo .dta")
paper_reference_file <- here("prev_analysis", "667.full.pdf")
outcome_labels <- c(
  alive = "Alive at age 5",
  died = "Died before age 5"
)

main_tree_control <- ci_tree_control(
  minsplit = 500L,
  minbucket = 250L,
  minprob = 0.02,
  maxdepth = 5L,
  min_gain = 0.00001,
  min_relative_gain = 0.20
)

```

```

map_model_terms_to_predictors <- function(term_names, predictors) {
  vapply(term_names, function(term) {
    hit <- predictors[term == predictors | startsWith(term, predictors)]
    c(hit[which.max(nchar(hit))], term)[1L]
  }, character(1L))
}

```

```
}
```

```
canonicalize_model_terms <- function(term_names, reference_terms) {  
  lookup <- data.table(  
    variable = reference_terms,  
    syntactic = make.names(reference_terms, unique = FALSE)  
  )  
  lookup <- lookup[  
    !duplicated(syntactic) & !duplicated(syntactic, fromLast = TRUE)  
  ]  
  
  out <- term_names  
  exact <- match(out, reference_terms)  
  out[!is.na(exact)] <- reference_terms[exact[!is.na(exact)]]  
  
  needs_lookup <- which(is.na(exact))  
  syntactic <- match(out[needs_lookup], lookup$syntactic)  
  out[needs_lookup[!is.na(syntactic)]] <- lookup$variable[syntactic[!is.na(syntactic)]]  
  
  out  
}
```

```
rank_from_named_vector <- function(x, method_name) {  
  x <- x %||% numeric()  
  data.table(variable = names(x), score = as.numeric(x))[  
    is.finite(score) & score != 0  
  ][  
    order(-abs(score))  
  ][  
    ,  
    .(variable, method = method_name, rank = seq_len(.N), score)  
  ]  
}
```

1 Purpose

This document is the canonical generator for the objects used by the main DRC results report. It is written as a readable workflow so a lecturer can follow what is loaded, what is fitted, what is compared, and what is finally saved.

If you need a plain R script for batch execution, extract it from this document with `knitr::purl("Generate_DRC_Results.R")`. Display-only chunks are excluded from the purl output so the extracted script stays focused on computation and saving.

2 Run Configuration

The runtime of this generator is controlled by two quantities: the number of analysis rows and the number of tuning folds. The remaining modelling choices are fixed below so the workflow stays readable. Set `analysis_rows` to 1 to use the full data set.

```
library(haven)  
drc_2007_dt <- as.data.table(haven::read_dta(source_data_file))
```

```

factor_from_codes <- function(x, labels) {
  factor(labels[as.character(as.integer(x))], levels = unname(labels))
}

binary_factor <- function(x, labels) {
  factor(labels[as.character(as.integer(x))], levels = unname(labels))
}

drc_birth_labels <- c(
  "1" = "First birth",
  "2" = "2-4 short interval",
  "3" = "2-4 long interval",
  "4" = "5+ short interval",
  "5" = "5+ long interval"
)

drc_wealth_group_labels <- c("1" = "High household wealth", "2" = "Low household wealth")
drc_age_mother_labels <- c("1" = "20 or more", "2" = "Less than 20")
drc_education_labels <- c("1" = "Any education", "2" = "No education")
drc_occupation_labels <- c(
  "1" = "Other",
  "2" = "Household, unskilled, not working",
  "3" = "Agriculture"
)

drc_region_labels <- c(
  "1" = "Bandundu",
  "2" = "Bas-Congo",
  "3" = "Equateur",
  "4" = "Kasai Occidental",
  "5" = "Kasai Oriental",
  "6" = "Katanga",
  "7" = "Kinshasa",
  "8" = "Maniema",
  "9" = "Nord-Kivu",
  "10" = "Orientale",
  "11" = "Sud-Kivu"
)

raw_congo_model_dt <- drc_2007_dt[
,
.(
  wealth = as.numeric(wealth),
  deadu5_num = as.integer(deadu5),
  sample_weight = as.numeric(weight),
  quint = factor_from_codes(quint, drc_wealth_group_labels),
  unskilled = binary_factor(
    unskilled,
    c("0" = "Skilled birth attendance", "1" = "No skilled birth attendance")
  ),
  male = binary_factor(male, c("0" = "Female", "1" = "Male")),
  birth = factor_from_codes(birth, drc_birth_labels),
  agemoth = factor_from_codes(agemoth, drc_age_mother_labels),
  rural = binary_factor(rural, c("0" = "Urban", "1" = "Rural")),
  ed = factor_from_codes(ed, drc_education_labels),
  ped = factor_from_codes(ped, drc_education_labels),
  mocc = factor_from_codes(mocc, drc_occupation_labels),
  pocc = factor_from_codes(pocc, drc_occupation_labels),
  reg = factor_from_codes(reg, drc_region_labels)
)

```

```

]
raw_congo_model_dt <- na.omit(raw_congo_model_dt)

raw_congo_var_labels <- c(
  quint = "Household wealth",
  unskilled = "Skilled birth attendance",
  male = "Sex of the child",
  birth = "Birth order and interval",
  agemoth = "Mother's age at birth",
  rural = "Type of residence",
  ed = "Mother's education",
  ped = "Father's education",
  mocc = "Mother's occupation",
  pocc = "Father's occupation",
  reg = "Region"
)
raw_congo_predictors <- names(raw_congo_var_labels)

```

```

raw_congo_model_dt <- sample_analysis_rows(
  raw_congo_model_dt,
  use_full = runtime_params$analysis_rows == 1L,
  n = runtime_params$analysis_rows,
  seed = 20260516
)

```

```

predictor_level_counts <- vapply(
  raw_congo_model_dt[, raw_congo_predictors, with = FALSE],
  function(x) length(levels(as.factor(x))),
  integer(1L)
)
nominal_predictors <- raw_congo_predictors[
  vapply(
    raw_congo_model_dt[, raw_congo_predictors, with = FALSE],
    function(x) is.factor(x) || is.character(x),
    logical(1L)
  )
]
multi_level_predictors <- intersect(
  names(predictor_level_counts[predictor_level_counts > 2L]),
  nominal_predictors
)
kept_predictors <- setdiff(raw_congo_predictors, multi_level_predictors)

make_indicator_name <- function(variable, level) {
  make.names(paste(variable, level, sep = "_"), unique = FALSE)
}

indicator_level_map <- rbindlist(lapply(multi_level_predictors, function(variable) {
  levels_i <- levels(as.factor(raw_congo_model_dt[[variable]]))
  data.table(
    raw_variable = variable,
    level = levels_i,
    variable = make_indicator_name(variable, levels_i),
    variable_label = paste0(unnamed(raw_congo_var_labels[variable]), ": ", levels_i)
  )
}), fill = TRUE)

```

```

stopifnot(!anyDuplicated(indicator_level_map$variable))

model_predictor_dt <- copy(raw_congo_model_dt[, kept_predictors, with = FALSE])
for (i in seq_len(nrow(indicator_level_map))) {
  source_variable <- indicator_level_map$raw_variable[i]
  source_level <- indicator_level_map$level[i]
  target_variable <- indicator_level_map$variable[i]
  model_predictor_dt[
    ,
    (target_variable) := factor(
      fifelse(as.character(raw_congo_model_dt[[source_variable]]) == source_level, "Yes", "No",
      levels = c("No", "Yes")
    )
  ]
}

congo_model_dt <- data.table(
  raw_congo_model_dt[, .(wealth, deadu5_num, sample_weight)],
  model_predictor_dt
)

congo_predictors <- names(model_predictor_dt)
congo_var_labels <- setNames(congo_predictors, congo_predictors)
congo_var_labels[intersect(kept_predictors, names(raw_congo_var_labels))] <-
  raw_congo_var_labels[intersect(kept_predictors, names(raw_congo_var_labels))]
congo_var_labels[indicator_level_map$variable] <- indicator_level_map$variable_label

congo_ci_formula <- as.formula(
  paste("cbind(wealth, deadu5_num) ~", paste(congo_predictors, collapse = " + "))
)

canonicalize_regression_terms <- function(term_names) {
  vapply(term_names, function(term) {
    parent <- map_model_terms_to_predictors(term, raw_congo_predictors)
    source_level <- sub(paste0("^", parent), "", term)
    hit <- indicator_level_map[raw_variable == parent & level == source_level]
    choices <- c(hit$variable[1L], parent, term)
    choice_id <- which(c(
      parent %in% multi_level_predictors && parent != term && nrow(hit) > 0L,
      parent %in% raw_congo_predictors && parent != term,
      TRUE
    ))[1L]
    choices[choice_id]
  }, character(1L))
}

format_variable_label <- function(x) {
  vapply(x, function(value) {
    parent <- map_model_terms_to_predictors(value, raw_congo_predictors)
    suffix <- trimws(sub(paste0("^", parent), "", value))
    suffix <- gsub("^[:. _]+", "", suffix)
    parent_label <- unname(raw_congo_var_labels[parent])
    choices <- c(
      NA_character_,
      unname(congo_var_labels[value]),
      unname(raw_congo_var_labels[value]),
      paste0(parent_label, ": ", suffix),
      parent_label,
      value
    )
  }, character(5L))
}

```

```

)
choice_id <- which(c(
  is.na(value) || !nzchar(value),
  value %in% names(congo_var_labels),
  value %in% names(raw_congo_var_labels),
  parent %in% names(raw_congo_var_labels) && parent != value && nzchar(suffix),
  parent %in% names(raw_congo_var_labels) && parent != value,
  TRUE
))[1L]
choices[choice_id]
}, character(1L))
}

```

```

report_compact_table(
  data.table(
    item = c(
      "Source data",
      "Rows requested",
      "Rows used",
      "Tuning folds",
      "Published benchmark",
      "Candidate predictors",
      "Output file"
    ),
    value = c(
      source_data_file,
      fifelse(
        runtime_params$analysis_rows == 1L,
        "Full data set",
        format(runtime_params$analysis_rows, big.mark = ",", scientific = FALSE)
      ),
      format(nrow(congo_model_dt), big.mark = ",", scientific = FALSE),
      runtime_params$tuning_folds,
      "Van Malderen et al. 2013 Table 3, DR Congo 2007",
      length(congo_predictors),
      results_object_file
    )
  ),
  caption = "Run configuration for the DRC results generator",
  column_widths = c("4.5cm", "9cm")
)

```

Table 1: Run configuration for the DRC results generator

item	value
Source data	C:/Users/moses.mburu.FIND/Pictures/personal/thesis-mse/prev_analysis/DR Congo .dta
Rows requested	Full data set
Rows used	8,296
Tuning folds	10
Published benchmark	Van Malderen et al. 2013 Table 3, DR Congo 2007
Candidate predictors	29
Output file	C:/Users/moses.mburu.FIND/Pictures/personal/thesis-mse/data/drc_report_results_objects.rda

3 Build Descriptive Objects

This section constructs the basic descriptive outputs that anchor the rest of the report: outcome prevalence, predictor coverage, and root-node inequality.

```
outcome_plot_dt <- congo_model_dt[
  ,
  .(weighted_n = sum(sample_weight)),
  by = .(deadu5_num)
][
  ,
  outcome_label := fifelse(deadu5_num == 1, outcome_labels[["died"]], outcome_labels[["alive"]])
][
  ,
  weighted_percent := 100 * weighted_n / sum(weighted_n)
]

candidate_predictor_table <- rbindlist(lapply(raw_congo_predictors, function(var) {
  x <- raw_congo_model_dt[[var]]
  level_dt <- raw_congo_model_dt[, .(weighted_n = sum(sample_weight)), by = var][order(-weighted_n)]
  level_dt[, weighted_percent := 100 * weighted_n / sum(weighted_n)]
  data.table(
    variable = var,
    label = unname(raw_congo_var_labels[var]),
    class = paste(class(x), collapse = "/"),
    n_levels = uniqueN(x),
    most_common_level = as.character(level_dt[[var]][1L]),
    most_common_percent = level_dt$weighted_percent[1L]
  )
}), fill = TRUE)

drc_root_ci_table <- rbindlist(lapply(criterion_types, function(type) {
  ci_fun <- ci_factory(type)
  data.table(
    criterion = type,
    estimate = ci_fun(
      cbind(congo_model_dt$wealth, congo_model_dt$deadu5_num),
      congo_model_dt$sample_weight
    )
  )
}))

drc_data_summary_table <- data.table(
  item = c(
    "Analysis rows",
    "Weighted sample size",
    "Weighted under-five mortality",
    "Ranking variable",
    "Candidate predictors"
  ),
  value = c(
    format(nrow(congo_model_dt), big.mark = ",", scientific = FALSE),
    format(round(sum(congo_model_dt$sample_weight)), big.mark = ",", scientific = FALSE),
    sprintf("%.2f%%", 100 * weighted_mean_safe(congo_model_dt$deadu5_num, congo_model_dt$sample_weight)),
    "DHS wealth index",
    paste(unname(raw_congo_var_labels), collapse = "; ")
  )
)
```



```
report_compact_table(
  drc_data_summary_table,
  caption = "Analysis sample summary used by the DRC results report",
  column_widths = c("4cm", "10cm")
)
```

Table 2: Analysis sample summary used by the DRC results report

item	value
Analysis rows	8,296
Weighted sample size	8,323
Weighted under-five mortality	10.32%
Ranking variable	DHS wealth index
Candidate predictors	Household wealth; Skilled birth attendance; Sex of the child; Birth order and interval; Mother's age at birth; Type of residence; Mother's education; Father's education; Mother's occupation; Father's occupation; Region

```
report_compact_table(
  drc_root_ci_table,
  digits = 4,
  caption = "Root-node concentration-index estimates across criteria"
)
```

Table 3: Root-node concentration-index estimates across criteria

criterion	estimate
CI	0.1156
CIg	0.0119
CIc	0.0477
L	0.1935

4 Build Classical Decomposition Objects

This section now follows a result-first workflow. The survey model is fitted in its own chunk, the summary is shown immediately after, and each `rineq` decomposition method is built in its own chunk before being displayed.

```
library(survey)
library(rineq)
linear_decomp_formula <- as.formula(
  paste("deadu5_num ~", paste(raw_congo_predictors, collapse = " + "))
)
linear_decomp_design <- survey::svydesign(
  ids = ~1,
  weights = ~sample_weight,
  data = raw_congo_model_dt
)
linear_decomp_fit <- survey::svyglm(
  linear_decomp_formula,
```

```
design = linear_decomp_design,
family = quasibinomial()
)
```

```
print(summary(linear_decomp_fit))
```

Call:

```
svyglm(formula = linear_decomp_formula, design = linear_decomp_design,
family = quasibinomial())
```

Survey design:

```
survey::svydesign(ids = ~1, weights = ~sample_weight, data = raw_congo_model_dt)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-2.908431	0.275345	-10.563	<2e-16 ***
quintLow household wealth	0.068117	0.125237	0.544	0.5865
unskilledNo skilled birth attendance	0.198131	0.111115	1.783	0.0746 .
maleMale	0.241990	0.105496	2.294	0.0218 *
birth2-4 short interval	-0.003766	0.185540	-0.020	0.9838
birth2-4 long interval	-0.509098	0.168982	-3.013	0.0026 **
birth5+ short interval	0.419653	0.210396	1.995	0.0461 *
birth5+ long interval	-0.396393	0.210666	-1.882	0.0599 .
agemothLess than 20	0.032723	0.142687	0.229	0.8186
ruralRural	0.090066	0.154181	0.584	0.5591
edNo education	0.336221	0.131606	2.555	0.0106 *
pedNo education	-0.035664	0.127535	-0.280	0.7798
moccHousehold, unskilled, not working	-0.017693	0.155378	-0.114	0.9093
moccAgriculture	0.140468	0.163233	0.861	0.3895
poccHousehold, unskilled, not working	0.235475	0.162445	1.450	0.1472
poccAgriculture	0.224936	0.149083	1.509	0.1314
regBas-Congo	0.258066	0.243006	1.062	0.2883
regEcuator	0.249695	0.237621	1.051	0.2934
regKasai Occidental	0.105100	0.228244	0.460	0.6452
regKasai Oriental	0.152682	0.240257	0.635	0.5251
regKatanga	0.515018	0.228125	2.258	0.0240 *
regKinshasa	0.081422	0.262411	0.310	0.7564
regManiema	0.488879	0.224476	2.178	0.0294 *
regNord-Kivu	-0.251522	0.293512	-0.857	0.3915
regOrientale	-0.083627	0.264827	-0.316	0.7522
regSud-Kivu	0.299991	0.270055	1.111	0.2667

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for quasibinomial family taken to be 1.013466)

Number of Fisher Scoring iterations: 5

```
linear_x <- model.matrix(linear_decomp_formula, data = raw_congo_model_dt)
linear_beta <- coef(linear_decomp_fit)
linear_terms <- setdiff(colnames(linear_x), "(Intercept)")

linear_term_info <- data.table(
  term = linear_terms,
  variable = canonicalize_regression_terms(linear_terms),
  linear_coefficient = unname(linear_beta[linear_terms]),
```

```

mean_of_variable = vapply(
  linear_terms,
  function(term) weighted_mean_safe(linear_x[, term], raw_congo_model_dt$sample_weight),
  numeric(1L)
)
)

regression_model_summary <- as.data.table(
  as.data.frame(broom::tidy(linear_decomp_fit, conf.int = TRUE))
)
regression_model_summary[, variable := fifelse(
  term == "(Intercept)",
  term,
  canonicalize_regression_terms(term)
)]
regression_model_summary[, term_label := fifelse(
  term == "(Intercept)",
  "Intercept",
  format_variable_label(variable)
)]
setcolororder(
  regression_model_summary,
  c("term", "term_label", "variable", setdiff(names(regression_model_summary), c("term", "term_label"))
)

```

```

decomposition_to_table <- function(object, digits = 4L) {
  data.table(
    term = names(object$rel_contribution),
    `Contribution (%)` = round(object$rel_contribution, digits),
    `Contribution (Abs)` = round(object$ci_contribution, digits),
    Elasticity = round(c(0, object$elasticities), digits),
    `Concentration Index` = c(NA_real_, round(object$partial_cis, digits)),
    `lower 5%` = c(NA_real_, round(object$confints[1, ], digits)),
    `upper 5%` = c(NA_real_, round(object$confints[2, ], digits)),
    Corrected = c("", ifelse(object$corrected_coefficients, "yes", "no"))
  )
}

```

```

summarise_rineq_decomposition <- function(object, method_name) {
  out <- merge(
    linear_term_info,
    decomposition_to_table(object, digits = 6L)[
      term != "residual",
      .(
        term,
        elasticity = Elasticity,
        concentration_index_of_variable = `Concentration Index`,
        regression_contribution = `Contribution (Abs)`,
        regression_pct_contribution = `Contribution (%)`
      )
    ],
    by = "term",
    all.y = TRUE
  )
  out[, variable := fifelse(!is.na(variable) & nzchar(variable), variable, term)]
  out[, `:=`(
    criterion = method_name,
    variable_label = format_variable_label(variable)
  )
}

```

```

)]

setcolororder(
  out,
  c(
    "variable",
    "variable_label",
    "criterion",
    "linear_coefficient",
    "mean_of_variable",
    "elasticity",
    "concentration_index_of_variable",
    "regression_contribution",
    "regression_pct_contribution"
  )
)
out[order(-abs(regression_pct_contribution))]
}

```

```

rineq_ci_decomp <- rineq::decomposition(
  outcome = raw_congo_model_dt$deadu5_num,
  betas = unname(linear_beta[linear_terms]),
  mm = linear_x[, linear_terms, drop = FALSE],
  ranker = raw_congo_model_dt$wealth,
  wt = raw_congo_model_dt$sample_weight,
  correction = TRUE,
  citype = "CI"
)
rineq_ci_summary <- decomposition_to_table(rineq_ci_decomp, digits = 4L)
classical_ci_table <- summarise_rineq_decomposition(rineq_ci_decomp, "CI")

```

```

report_compact_table(
  rineq_ci_summary,
  digits = 4,
  caption = "rineq decomposition summary: CI"
)

```

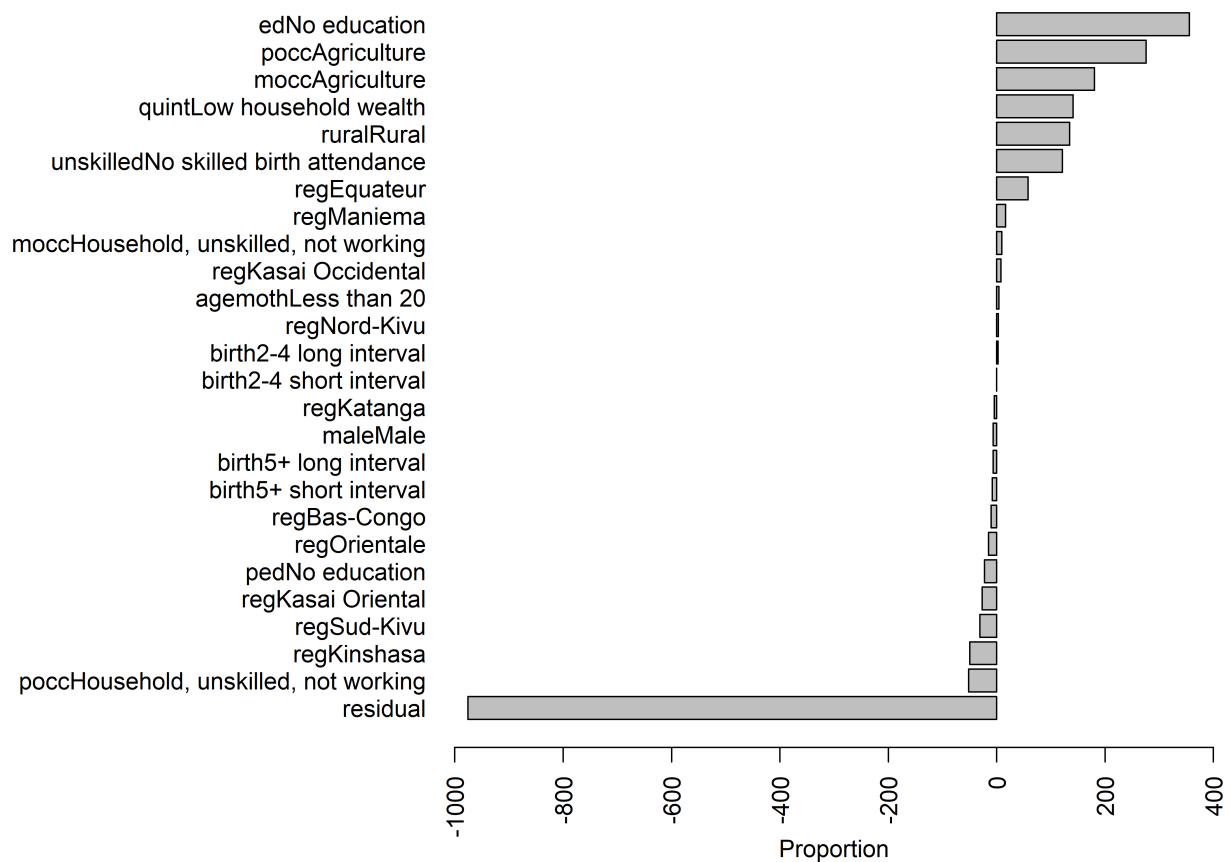
Table 4: rineq decomposition summary: CI

term	Contribution (%)	Contribution (Abs)	Elasticity	Concen
residual	-975.5357	1.1266	0.0000	
quintLow household wealth	140.6699	-0.1624	0.2885	
unskilledNo skilled birth attendance	121.0994	-0.1398	1.1080	
maleMale	-6.4425	0.0074	1.1564	
birth2-4 short interval	-0.0587	0.0001	-0.0039	
birth2-4 long interval	2.3826	-0.0028	-1.6719	
birth5+ short interval	-8.1400	0.0094	0.4140	
birth5+ long interval	-6.7469	0.0078	-1.0078	
agemothLess than 20	4.0288	-0.0047	0.1370	
ruralRural	134.7545	-0.1556	0.5377	
edNo education	355.7900	-0.4109	1.8406	
pedNo education	-22.5539	0.0260	-0.0932	
moccHousehold, unskilled, not working	9.2447	-0.0107	-0.0397	
moccAgriculture	180.2946	-0.2082	0.7603	

poccHousehold, unskilled, not working	-52.0012	0.0601	0.2933
poccAgriculture	275.6313	-0.3183	1.1003
regBas-Congo	-10.4632	0.0121	0.1003
regEquateur	58.0391	-0.0670	0.3108
regKasai Occidental	7.5561	-0.0087	0.1083
regKasai Oriental	-26.9466	0.0311	0.1970
regKatanga	-4.5699	0.0053	0.5145
regKinshasa	-49.7000	0.0574	0.0719
regManiema	16.5843	-0.0192	0.1668
regNord-Kivu	3.2146	-0.0037	-0.0949
regOrientale	-15.1820	0.0175	-0.0936
regSud-Kivu	-30.9490	0.0357	0.1400

```
plot(
  rineq_ci_decomp,
  horiz = TRUE,
  decreasing = FALSE,
  main = "rineq decomposition contributions: CI"
)
```

rineq decomposition contributions: CI



```
rineq_cig_decomp <- rineq::decomposition(
  outcome = raw_congo_model_dt$deadu5_num,
  betas = unname(linear_beta[linear_terms]),
  mm = linear_x[, linear_terms, drop = FALSE],
```

```

ranker = raw_congo_model_dt$wealth,
wt = raw_congo_model_dt$sample_weight,
correction = TRUE,
citype = "CIg"
)
rineq_cig_summary <- decomposition_to_table(rineq_cig_decomp, digits = 4L)
classical_cig_table <- summarise_rineq_decomposition(rineq_cig_decomp, "CIg")

report_compact_table(
  rineq_cig_summary,
  digits = 4,
  caption = "rineq decomposition summary: CIg"
)

```

Table 5: rineq decomposition summary: CIg

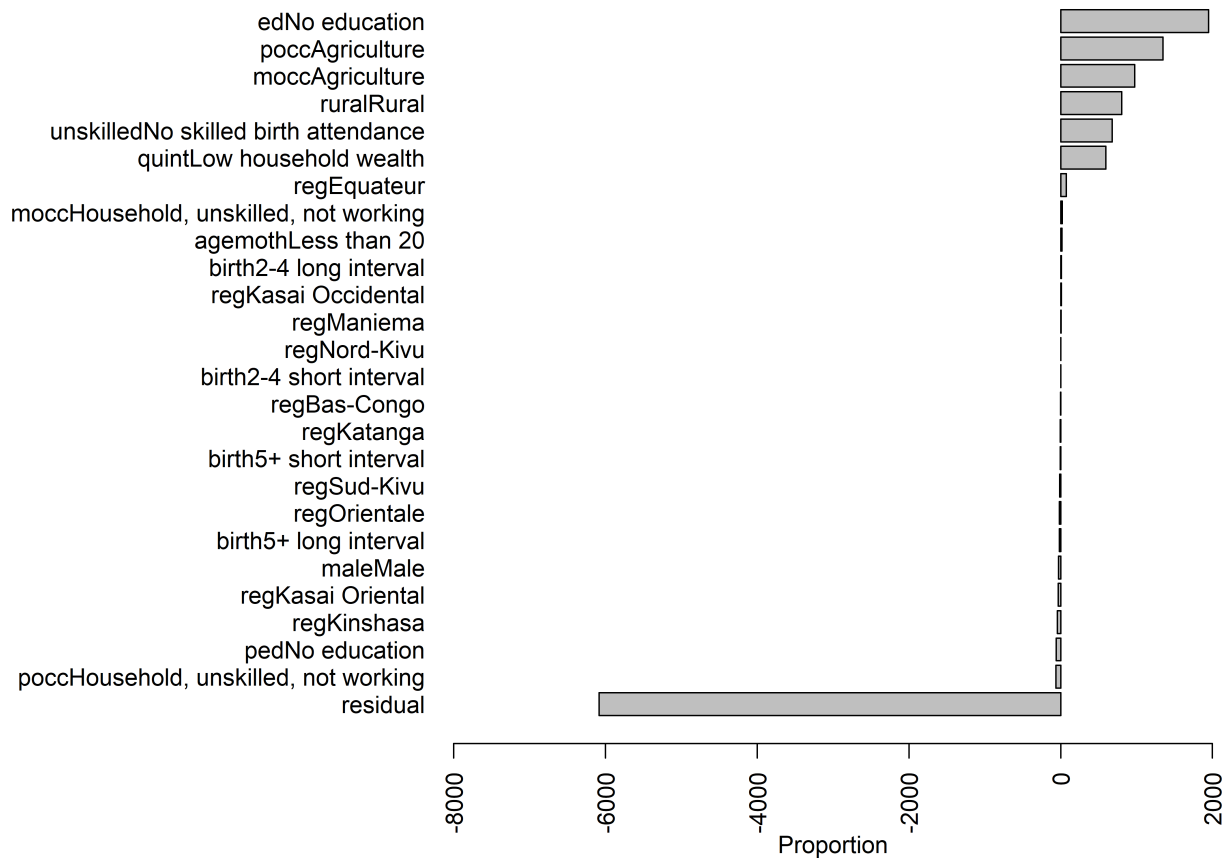
term	Contribution (%)	Contribution (Abs)	Elasticity	Concen
residual	-6083.3897	0.7249	0.0000	
quintLow household wealth	595.8302	-0.0710	0.2885	
unskilledNo skilled birth attendance	677.2228	-0.0807	1.1080	
maleMale	-30.7864	0.0037	1.1564	
birth2-4 short interval	-0.0605	0.0000	-0.0039	
birth2-4 long interval	7.8246	-0.0009	-1.6719	
birth5+ short interval	-8.0306	0.0010	0.4140	
birth5+ long interval	-17.1537	0.0020	-1.0078	
agemothLess than 20	16.8616	-0.0020	0.1370	
ruralRural	804.5451	-0.0959	0.5377	
edNo education	1947.7168	-0.2321	1.8406	
pedNo education	-58.9303	0.0070	-0.0932	
moccHousehold, unskilled, not working	20.7690	-0.0025	-0.0397	
moccAgriculture	975.8716	-0.1163	0.7603	
poccHousehold, unskilled, not working	-64.7787	0.0077	0.2933	
poccAgriculture	1348.2802	-0.1607	1.1003	
regBas-Congo	-4.0657	0.0005	0.1003	
regEquateur	72.2531	-0.0086	0.3108	
regKasai Occidental	7.7831	-0.0009	0.1083	
regKasai Oriental	-34.7631	0.0041	0.1970	
regKatanga	-4.5651	0.0005	0.5145	
regKinshasa	-43.8701	0.0052	0.0719	
regManiema	5.6569	-0.0007	0.1668	
regNord-Kivu	1.2135	-0.0001	-0.0949	
regOrientale	-16.9917	0.0020	-0.0936	
regSud-Kivu	-14.4431	0.0017	0.1400	

```

plot(
  rineq_cig_decomp,
  horiz = TRUE,
  decreasing = FALSE,
  main = "rineq decomposition contributions: CIg"
)

```

rineq decomposition contributions: CIg



```
rineq_cic_decomp <- rineq::decomposition(
  outcome = raw_congo_model_dt$deadu5_num,
  betas = unname(linear_beta[linear_terms]),
  mm = linear_x[, linear_terms, drop = FALSE],
  ranker = raw_congo_model_dt$wealth,
  wt = raw_congo_model_dt$sample_weight,
  correction = TRUE,
  citype = "CIc"
)
rineq_cic_summary <- decomposition_to_table(rineq_cic_decomp, digits = 4L)
classical_cic_table <- summarise_rineq_decomposition(rineq_cic_decomp, "CIc")

report_compact_table(
  rineq_cic_summary,
  digits = 4,
  caption = "rineq decomposition summary: CIc"
)
```

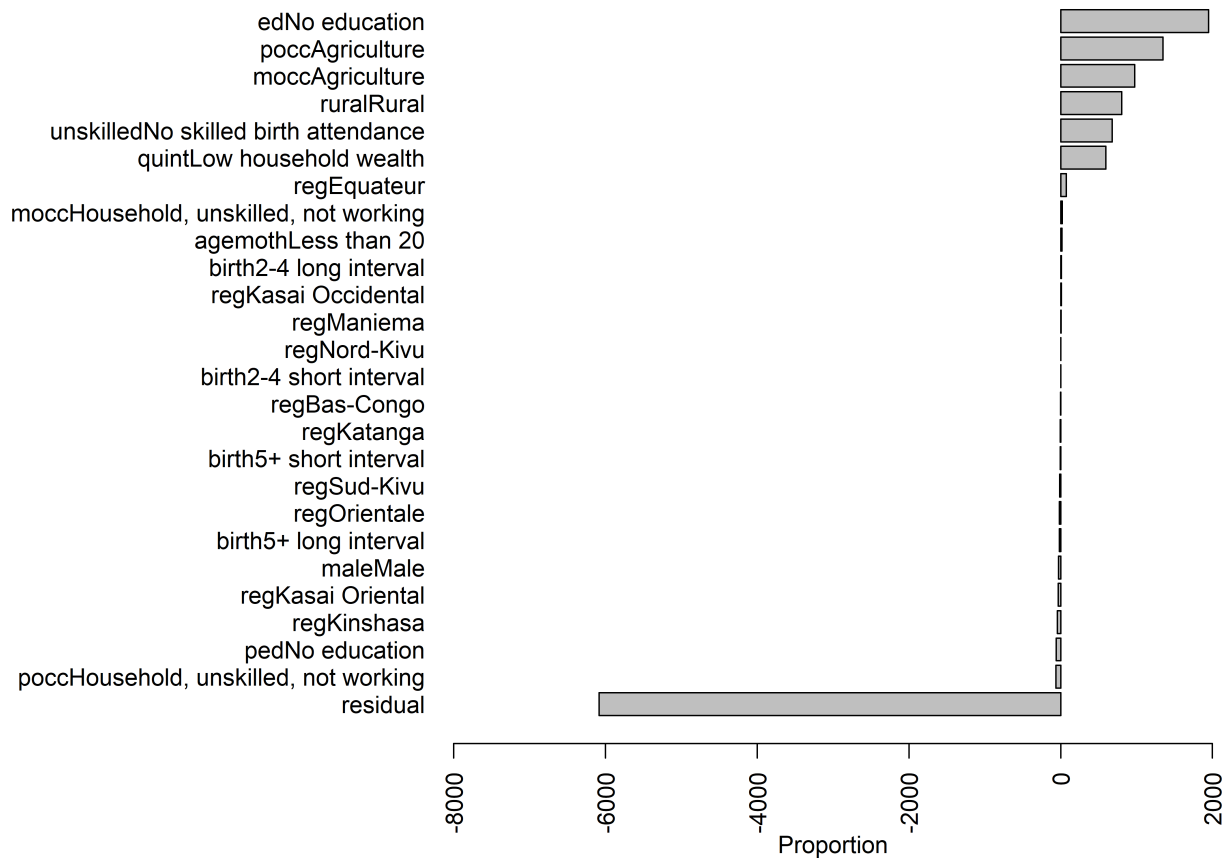
Table 6: rineq decomposition summary: CIc

term	Contribution (%)	Contribution (Abs)	Elasticity	Concen
residual	-6083.3897	2.8995	0.0000	
quintLow household wealth	595.8302	-0.2840	0.2885	
unskilledNo skilled birth attendance	677.2228	-0.3228	1.1080	
maleMale	-30.7864	0.0147	1.1564	

birth2-4 short interval	-0.0605	0.0000	-0.0039
birth2-4 long interval	7.8246	-0.0037	-1.6719
birth5+ short interval	-8.0306	0.0038	0.4140
birth5+ long interval	-17.1537	0.0082	-1.0078
agemothLess than 20	16.8616	-0.0080	0.1370
ruralRural	804.5451	-0.3835	0.5377
edNo education	1947.7168	-0.9283	1.8406
pedNo education	-58.9303	0.0281	-0.0932
moccHousehold, unskilled, not working	20.7690	-0.0099	-0.0397
moccAgriculture	975.8716	-0.4651	0.7603
poccHousehold, unskilled, not working	-64.7787	0.0309	0.2933
poccAgriculture	1348.2802	-0.6426	1.1003
regBas-Congo	-4.0657	0.0019	0.1003
regEquateur	72.2531	-0.0344	0.3108
regKasai Occidental	7.7831	-0.0037	0.1083
regKasai Oriental	-34.7631	0.0166	0.1970
regKatanga	-4.5651	0.0022	0.5145
regKinshasa	-43.8701	0.0209	0.0719
regManiema	5.6569	-0.0027	0.1668
regNord-Kivu	1.2135	-0.0006	-0.0949
regOrientale	-16.9917	0.0081	-0.0936
regSud-Kivu	-14.4431	0.0069	0.1400

```
plot(
  rineq_cic_decomp,
  horiz = TRUE,
  decreasing = FALSE,
  main = "rineq decomposition contributions: CIc"
)
```


rineq decomposition contributions: CIc



```
rineq_ciw_decomp <- rineq::decomposition(
  outcome = raw_congo_model_dt$deadu5_num,
  betas = unname(linear_beta[linear_terms]),
  mm = linear_x[, linear_terms, drop = FALSE],
  ranker = raw_congo_model_dt$wealth,
  wt = raw_congo_model_dt$sample_weight,
  correction = TRUE,
  citype = "CIw"
)
rineq_ciw_summary <- decomposition_to_table(rineq_ciw_decomp, digits = 4L)
classical_ciw_table <- summarise_rineq_decomposition(rineq_ciw_decomp, "CIw")

report_compact_table(
  rineq_ciw_summary,
  digits = 4,
  caption = "rineq decomposition summary: CIw"
)
```

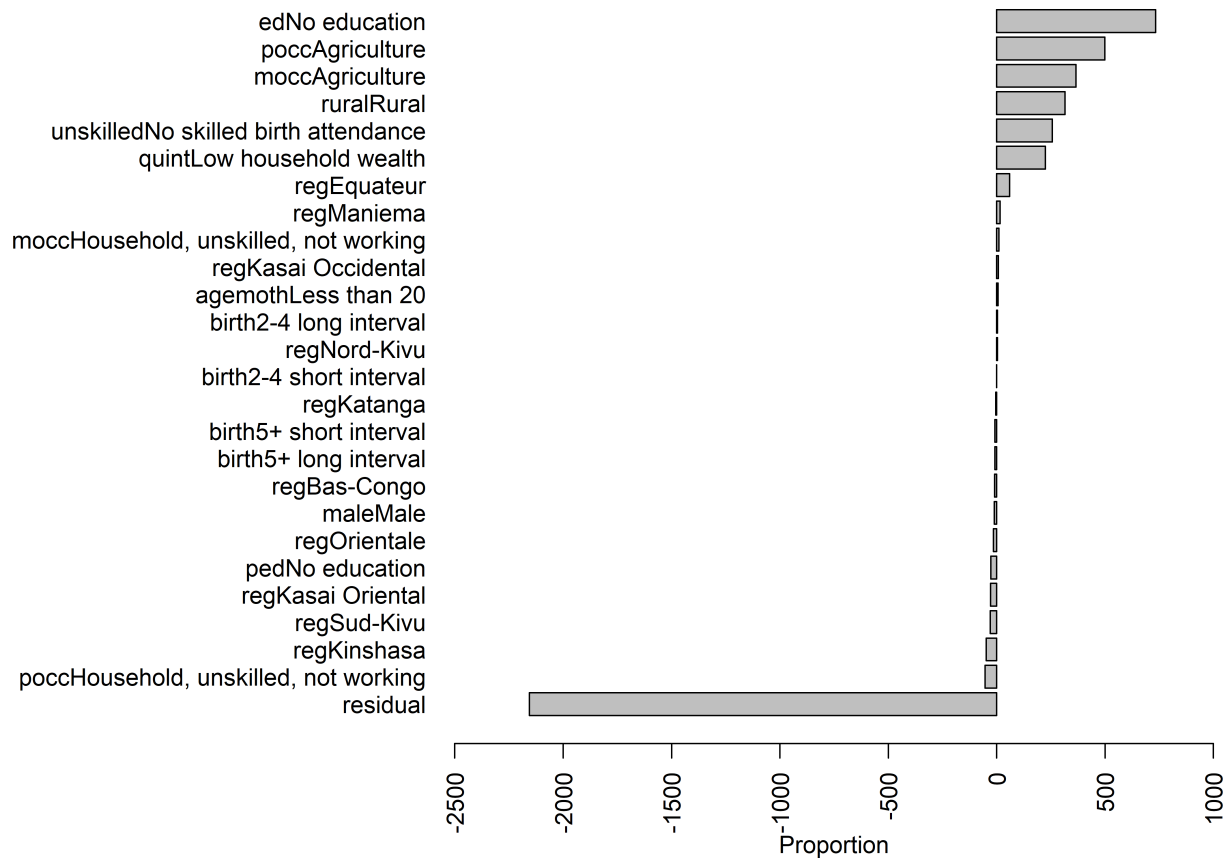
Table 7: rineq decomposition summary: CIw

term	Contribution (%)	Contribution (Abs)	Elasticity	Concen
residual	-2155.4990	2.7756	0.0000	
quintLow household wealth	224.0948	-0.2886	0.2885	
unskilledNo skilled birth attendance	256.7624	-0.3306	1.1080	
maleMale	-11.3976	0.0147	1.1564	

birth2-4 short interval	-0.0589	0.0001	-0.0039
birth2-4 long interval	3.2319	-0.0042	-1.6719
birth5+ short interval	-8.1275	0.0105	0.4140
birth5+ long interval	-8.2025	0.0106	-1.0078
agemothLess than 20	6.3594	-0.0082	0.1370
ruralRural	314.7506	-0.4053	0.5377
edNo education	733.2702	-0.9442	1.8406
pedNo education	-27.6927	0.0357	-0.0932
moccHousehold, unskilled, not working	10.7926	-0.0139	-0.0397
moccAgriculture	366.2231	-0.4716	0.7603
poccHousehold, unskilled, not working	-53.5141	0.0689	0.2933
poccAgriculture	499.1007	-0.6427	1.1003
regBas-Congo	-9.7755	0.0126	0.1003
regEquateur	59.7219	-0.0769	0.3108
regKasai Occidental	7.5823	-0.0098	0.1083
regKasai Oriental	-27.8770	0.0359	0.1970
regKatanga	-4.5694	0.0059	0.5145
regKinshasa	-49.0382	0.0631	0.0719
regManiema	15.4156	-0.0199	0.1668
regNord-Kivu	2.9997	-0.0039	-0.0949
regOrientale	-15.3931	0.0198	-0.0936
regSud-Kivu	-29.1597	0.0375	0.1400

```
plot(
  rineq_ciw_decomp,
  horiz = TRUE,
  decreasing = FALSE,
  main = "rineq decomposition contributions: CIw"
)
```

rineq decomposition contributions: CIw



```
rineq_decomp_by_type <- list(
  CI = rineq_ci_decomp,
  CIg = rineq_cig_decomp,
  CIc = rineq_cic_decomp,
  CIw = rineq_ciw_decomp
)

rineq_summary_by_type <- list(
  CI = rineq_ci_summary,
  CIg = rineq_cig_summary,
  CIc = rineq_cic_summary,
  CIw = rineq_ciw_summary
)

rineq_summary <- rbindlist(lapply(names(rineq_summary_by_type), function(type) {
  out <- copy(rineq_summary_by_type[[type]])
  out[, criterion := type]
  setcolororder(out, c("criterion", setdiff(names(out), "criterion")))
  out
}), fill = TRUE)

classical_decomposition_table <- rbindlist(
  list(classical_ci_table, classical_cig_table, classical_cic_table, classical_ciw_table),
  fill = TRUE
)
```

```
published_drc_table3_reference <- rbindlist(list(
  data.table(
    criterion = "G",
    determinant = c(
      "Household wealth",
      "Skilled birth attendance",
      "Sex of the child",
      "Birth order and interval",
      "Mother's age at birth",
      "Type of residence",
      "Mother's education",
      "Father's education",
      "Mother's occupation",
      "Father's occupation",
      "Region"
    ),
    published_index = -0.128,
    published_percent = c(2.4, 7.3, 5.3, 35.6, 0.4, 3.4, 17.4, -0.7, 5.8, 9.1, 14.0)
  ),
  data.table(
    criterion = "C",
    determinant = c(
      "Household wealth",
      "Skilled birth attendance",
      "Sex of the child",
      "Birth order and interval",
      "Mother's age at birth",
      "Type of residence",
      "Mother's education",
      "Father's education",
      "Mother's occupation",
      "Father's occupation",
      "Region"
    ),
    published_index = 0.057,
    published_percent = c(13.1, 11.3, -0.6, -1.2, 0.4, 12.5, 33.1, -2.1, 17.6, 20.8, -4.9)
  )
))

determinant_map <- data.table(
  variable = c("quint", "unskilled", "male", "birth", "agemoth", "rural", "ed", "ped", "mocc", "pocc"),
  raw_variable = c("quint", "unskilled", "male", "birth", "agemoth", "rural", "ed", "ped", "mocc", "pocc"),
  determinant = c(
    "Household wealth",
    "Skilled birth attendance",
    "Sex of the child",
    "Birth order and interval",
    "Mother's age at birth",
    "Type of residence",
    "Mother's education",
    "Father's education",
    "Mother's occupation",
    "Father's occupation",
    "Region"
  )
)

determinant_map <- rbindlist(
  list(
```

```

    determinant_map,
    merge(
      indicator_level_map[, .(variable, raw_variable)],
      determinant_map[, .(raw_variable, determinant)],
      by = "raw_variable",
      all.x = TRUE,
      sort = FALSE
    )[, .(variable, raw_variable, determinant)]
  ),
  fill = TRUE
)

classical_decomposition_grouped <- classical_decomposition_table[
  determinant_map,
  on = "variable",
  nomatch = OL
][
  ,
  .(
    regression_contribution = sum(regression_contribution, na.rm = TRUE),
    regression_pct_contribution = sum(regression_pct_contribution, na.rm = TRUE)
  ),
  by = .(criterion, determinant)
]

published_drc_table3_comparison <- merge(
  published_drc_table3_reference[criterion == "C"],
  classical_decomposition_grouped[criterion == "CI"],
  by = "determinant",
  all.x = TRUE,
  sort = FALSE
)

published_drc_table3_comparison[
  ,
  difference_from_published := regression_pct_contribution - published_percent
]

```

```

report_compact_table(
  published_drc_table3_comparison[
    ,
    .(
      determinant,
      paper_c_percent = published_percent,
      generated_ci_percent = regression_pct_contribution,
      difference_from_published
    )
  ],
  digits = 2,
  caption = "Published DR Congo 2007 concentration-index decomposition compared with generated regression decomposition"
)

```

Table 8: Published DR Congo 2007 concentration-index decomposition compared with generated regression decomposition

determinant	paper_c_percent	generated_ci_percent	difference_from_published
Household wealth	13.1	140.67	127.57

Skilled birth attendance	11.3	121.10	109.80
Sex of the child	-0.6	-6.44	-5.84
Birth order and interval	-1.2	-12.56	-11.36
Mother's age at birth	0.4	4.03	3.63
Type of residence	12.5	134.75	122.25
Mother's education	33.1	355.79	322.69
Father's education	-2.1	-22.55	-20.45
Mother's occupation	17.6	189.54	171.94
Father's occupation	20.8	223.63	202.83
Region	-4.9	-52.42	-47.52

5 Fit Concentration-Index Trees

This is the core generator step. The grid is cross-validated, then each criterion is refitted at its selected setting.

```
tree_grid <- ci_tree_control_grid(
  minsplit = c(500L), # 750L worked
  minbucket = c(100L), # 100 worked
  minprob = 0.01,
  maxdepth = 10L,
  min_gain = 0.0001,
  min_relative_gain = c(0.15, 0.20, 0.25, 0.30)
)

# filter where minbucket is at least 50% of minsplit to avoid invalid settings
tree_grid <- tree_grid[minbucket <= 0.5 * minsplit]
```

```
ci_tuning_metrics <- c(
  "validation_gain",
  "relative_validation_gain",
  "percent_validation_root_recovered",
  "brier",
  "log_loss",
  "roc_auc"
)

# tree_tuning <- tune_ci_tree(
#   formula = congo_ci_formula,
#   data = congo_model_dt,
#   rank_name = "wealth",
#   outcome_name = "deadu5_num",
#   weights = congo_model_dt$sample_weight,
#   type = criterion_types,
#   control_grid = tree_grid,
#   v = runtime_params$tuning_folds,
#   strata = "deadu5_num",
#   seed = 20260508,
#   metric = ci_tuning_metrics,
#   refit = FALSE,
#   control = control_ci_tune(save_pred = TRUE)
# )
```

```
# save(tree_tuning, file = here("data", "tree_tuning.rda"))
load(here("data", "tree_tuning.rda"))
```

```
tree_best_by_type <- ci_select_best(tree_tuning,
  metric = "relative_validation_gain"
)
tree_selection_table <- ci_fit_summary_table(
  tree_tuning,
  selected = tree_best_by_type,
  metrics = unique(tree_tuning$summary$metric),
  include_percent = FALSE
)
tree_selection_table[
  tree_best_by_type,
  terminal_nodes := i.mean_terminal_nodes,
  on = c("type", "grid_id")
]
tree_selection_table[, selection_basis := paste(
  runtime_params$tuning_folds,
  "fold cross-validation"
)]
report_compact_table(
  tree_selection_table,
  digits = 4,
  caption = "Summary of the concentration-index tree selection results across criteria"
)
```

type	grid_id	minsplit	minbucket	minprob	maxdepth	min_gain	min_relative_gain	n
CI	4	500	100	0.01	10	1e-04	0.3	
CIc	12	500	100	0.01	10	1e-04	0.3	
CIg	8	500	100	0.01	10	1e-04	0.3	
L	16	500	100	0.01	10	1e-04	0.3	

```
tree_models_by_type <- setNames(
  lapply(seq_len(nrow(tree_best_by_type)), function(i) {
    setting <- tree_best_by_type[i]
    ci_tree(
      formula = congo_ci_formula,
      data = congo_model_dt,
      rank_name = "wealth",
      outcome_name = "deadu5_num",
      weights = congo_model_dt$sample_weight,
      type = setting$type[1L],
      control = ci_control_from_row(setting)
    )
  }),
  tree_best_by_type$type
)
```

```
selected_tree_type <- tree_selection_table[order(-mean_validation_relative_gain)][1L, type]
selected_tree_fit <- tree_models_by_type[[selected_tree_type]]
selected_tree_summary <- as.data.table(ci_tree_terminal_summary(selected_tree_fit))
```

```
all_tree_terminal_summaries <- rbindlist(lapply(names(tree_models_by_type), function(type) {
  out <- as.data.table(ci_tree_terminal_summary(tree_models_by_type[[type]]))
  out[, criterion := type]
  setcolorder(out, "criterion")
  out
}), fill = TRUE)
```

```
tree_variable_importance <- collect_variable_importance(tree_models_by_type)

tree_best_controls <- tree_best_by_type[
  ,
  .(type, minsplit, minbucket, minprob, maxdepth, min_gain, min_relative_gain)
]

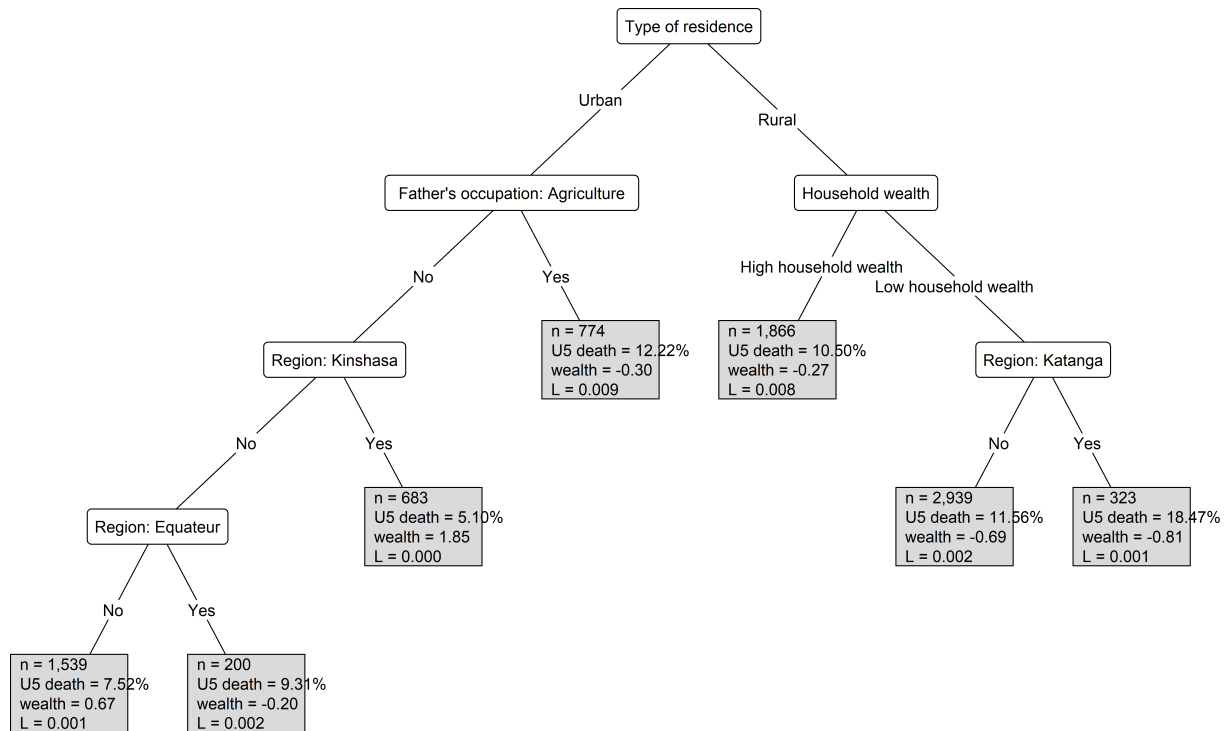
tree_complexity_metrics <- collect_complexity_metrics(tree_tuning)
tree_min_relative_gain_path <- min_relative_gain_path(tree_complexity_metrics,
  selected = tree_best_by_type
)

tree_fit_wide <- ci_fit_summary_table(
  tree_tuning,
  selected = tree_best_by_type,
  metrics = unique(tree_tuning$summary$metric),
  include_percent = FALSE
)
tree_fit_table <- ci_collect_metrics(
  tree_tuning,
  selected = tree_best_by_type,
  metric = unique(tree_tuning$summary$metric),
  format = "tidy",
  include_train = TRUE
)
```

```
report_compact_table(
  tree_selection_table,
  digits = 4,
  caption = "Selection summary for the fitted concentration-index trees"
)
```

type	grid_id	minsplit	minbucket	minprob	maxdepth	min_gain	min_relative_gain	n
CI	4	500	100	0.01	10	1e-04	0.3	
CIc	12	500	100	0.01	10	1e-04	0.3	
CIg	8	500	100	0.01	10	1e-04	0.3	
L	16	500	100	0.01	10	1e-04	0.3	

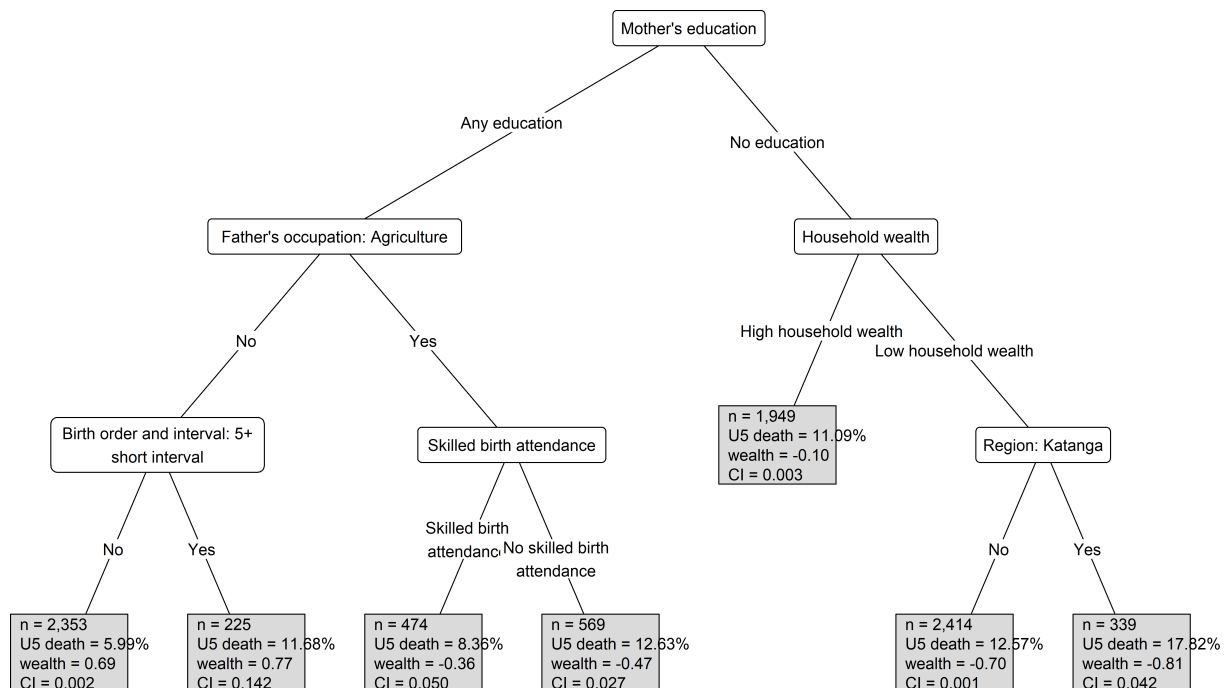
```
ci_report_tree_plot(
  fit = selected_tree_fit,
  data = congo_model_dt,
  outcome_name = "deadu5_num",
  outcome_label = "U5 death",
  ci_type = "L",
  var_labels = congo_var_labels
)
```

```

ci_report_tree_plot(
  fit = tree_models_by_type[["CI"]],
  data = congo_model_dt,
  outcome_name = "deadu5_num",
  outcome_label = "U5 death",
  ci_type = "CI",
  var_labels = congo_var_labels
)

```



```

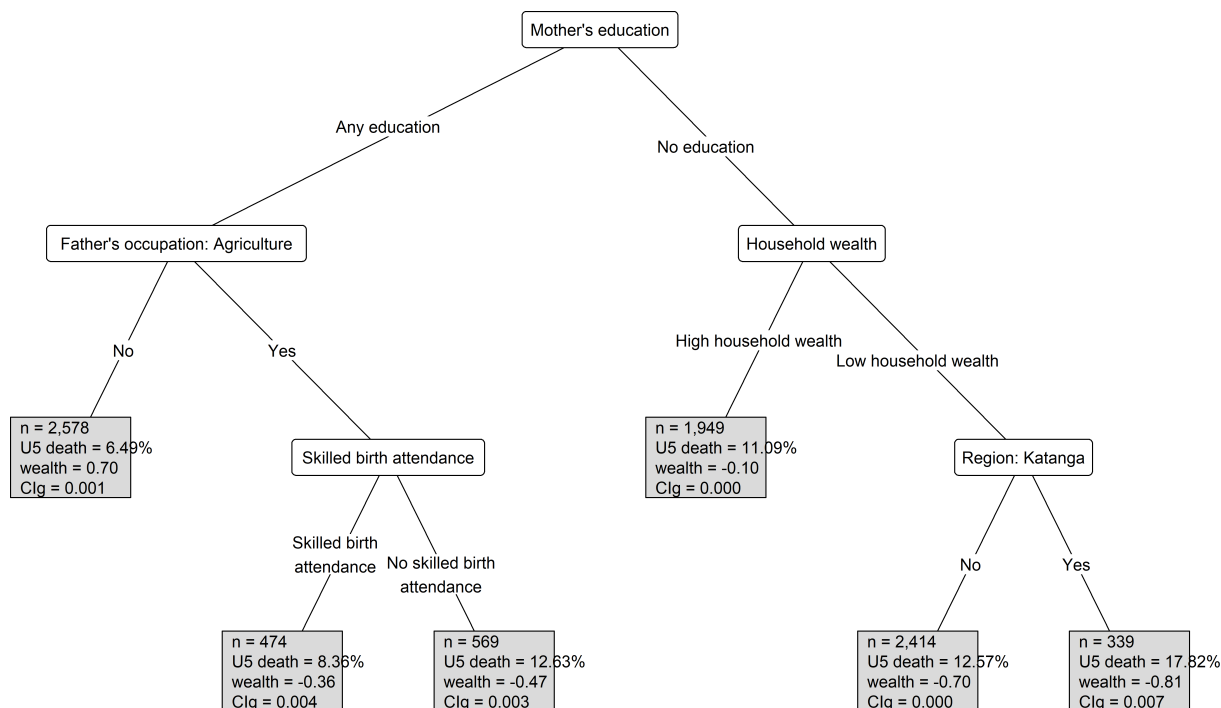
ci_report_tree_plot(
  fit = tree_models_by_type[["CIg"]],
  data = congo_model_dt,

```

```

outcome_name = "deadu5_num",
outcome_label = "U5 death",
ci_type = "CIg",
var_labels = congo_var_labels
)

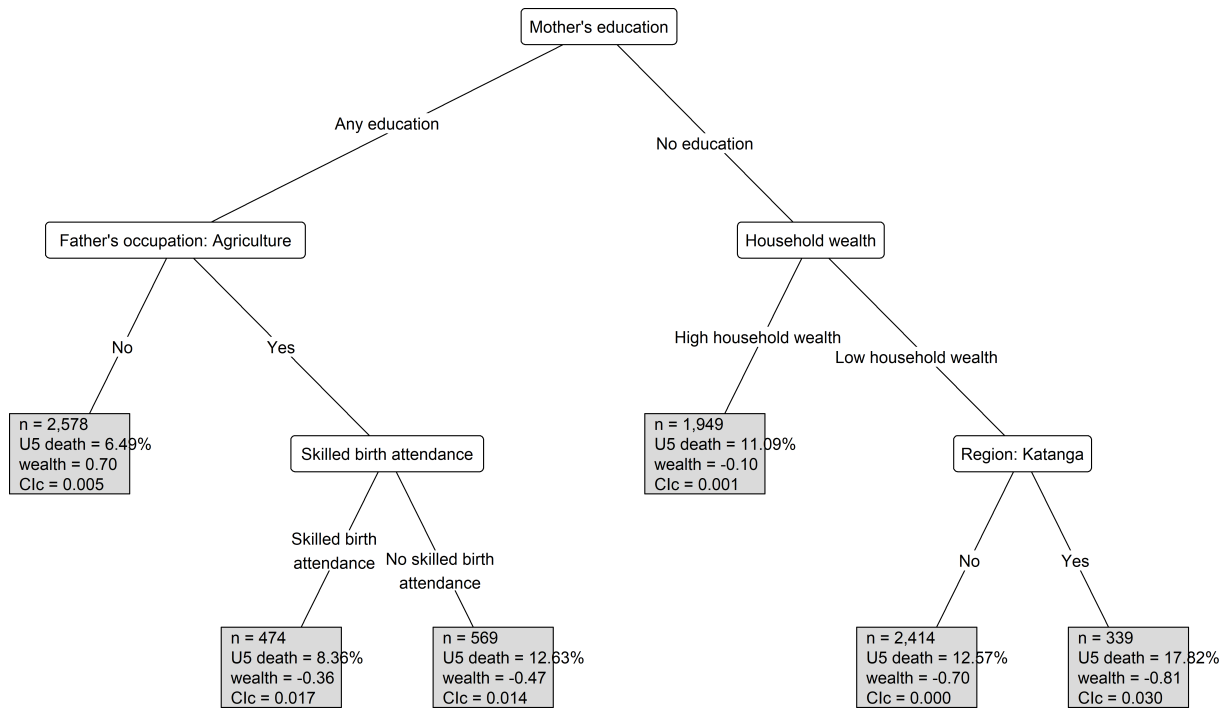
```



```

ci_report_tree_plot(
  fit = tree_models_by_type[["CIc"]],
  data = congo_model_dt,
  outcome_name = "deadu5_num",
  outcome_label = "U5 death",
  ci_type = "CIc",
  var_labels = congo_var_labels
)

```



6 Fit Lecturer Rpart Comparison Trees

The benchmark rpart trees are retained because they are easy to discuss beside the inequality-driven tree.

```

lecturer_rpart_models <- fit_ci_rpart_comparison(
  data = congo_model_dt,
  predictors = congo_predictors,
  outcome = "deadu5_num",
  rank = "wealth",
  weights = "sample_weight"
)
lecturer_rpart_summary <- summarise_ci_rpart_comparison(lecturer_rpart_models)

```

```

rpart_summary_out <- copy(lecturer_rpart_summary)[is_terminal == TRUE]

report_compact_table(
  rpart_summary_out,
  digits = 4,
  caption = "Terminal-node summaries for the lecturer rpart comparison trees"
)

```

Table 11: Terminal-node summaries for the lecturer rpart comparison trees

method	node	var	n	weight	deviance	mean_y	CI	CI_loss	is_terminal
simple_abs	16	<leaf>	907	926.0304	1.9375	0.0515	0.0021	0.0021	TRUE
simple_abs	17	<leaf>	1400	1571.0357	5.7339	0.0672	0.0036	0.0036	TRUE
simple_abs	18	<leaf>	43	18.3142	2.9358	0.1029	0.1603	0.1603	TRUE
simple_abs	38	<leaf>	14	5.4715	0.0000	0.0000	0.0000	0.0000	TRUE
simple_abs	78	<leaf>	7	2.8597	0.0000	0.0000	0.0000	0.0000	TRUE
simple_abs	79	<leaf>	120	54.3952	3.6348	0.2231	0.0668	0.0668	TRUE
simple_abs	10	<leaf>	68	86.5707	23.9450	0.1015	-0.2766	0.2766	TRUE

simple_abs	22	<leaf>	775	849.0024	5.0031	0.1014	-0.0059	0.0059	TRUE
simple_abs	23	<leaf>	77	108.0709	0.6296	0.1540	0.0058	0.0058	TRUE
simple_abs	6	<leaf>	2013	1948.8103	5.3891	0.1109	-0.0028	0.0028	TRUE
simple_abs	28	<leaf>	6	6.6534	0.0000	0.0000	0.0000	0.0000	TRUE
simple_abs	29	<leaf>	2512	2407.0755	0.3082	0.1260	-0.0001	0.0001	TRUE
simple_abs	30	<leaf>	162	145.1661	0.1906	0.1444	-0.0013	0.0013	TRUE
simple_abs	62	<leaf>	62	62.1253	0.4429	0.1359	0.0071	0.0071	TRUE
simple_abs	63	<leaf>	130	131.5099	8.6060	0.2354	-0.0654	0.0654	TRUE
robust_abs	8	<leaf>	1961	2025.0841	30.8769	0.0675	-0.0152	0.0152	TRUE
robust_abs	9	<leaf>	530	553.0227	8.0311	0.0552	-0.0145	0.0145	TRUE
robust_abs	10	<leaf>	374	474.3692	23.8352	0.0836	-0.0502	0.0502	TRUE
robust_abs	11	<leaf>	546	569.2748	15.4815	0.1263	-0.0272	0.0272	TRUE
robust_abs	6	<leaf>	2013	1948.8103	6.1032	0.1109	-0.0031	0.0031	TRUE
robust_abs	14	<leaf>	2518	2413.7289	1.4054	0.1257	0.0006	0.0006	TRUE
robust_abs	15	<leaf>	354	338.8012	14.2296	0.1782	-0.0420	0.0420	TRUE

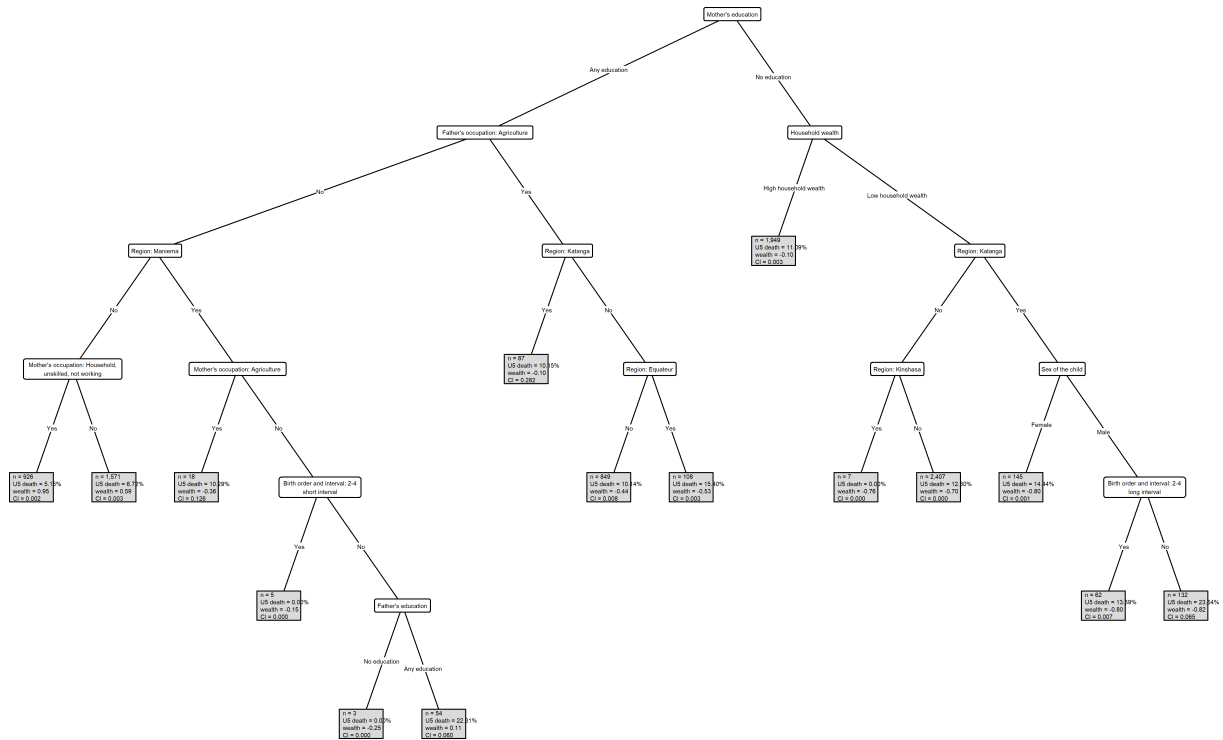
```

plot_labels <- c(
  simple_abs = "Lecturer rpart method 1: absolute CI impurity",
  robust_abs = "Lecturer rpart method 2: robust absolute CI impurity"
)
plot_fontsize <- c(simple_abs = 4.2, robust_abs = 4.6)

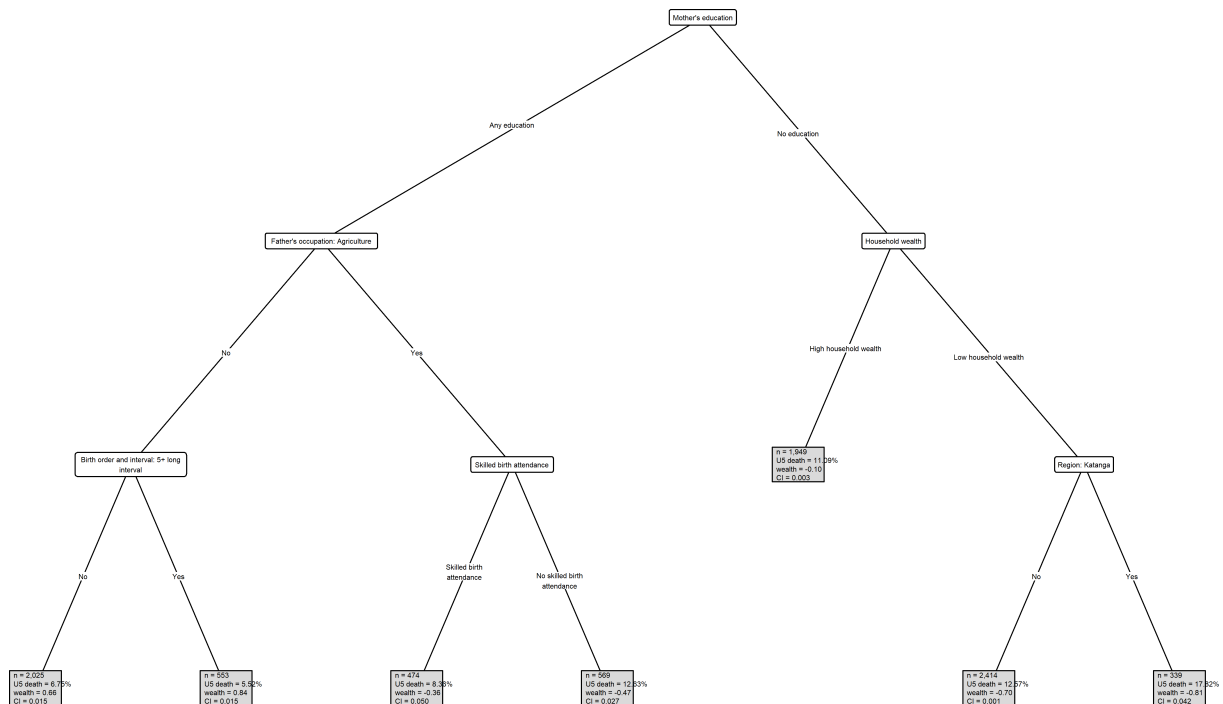
for (method_name in names(lecturer_rpart_models)) {
  cat("\n\n### ", plot_labels[[method_name]] %||% method_name, "\n\n", sep = "")
  ci_report_tree_plot(
    fit = lecturer_rpart_models[[method_name]],
    data = congo_model_dt,
    outcome_name = "deadu5_num",
    outcome_label = "U5 death",
    ci_type = "CI",
    var_labels = congo_var_labels,
    fontsize = plot_fontsize[[method_name]]
  )
}

```

6.0.1 Lecturer rpart method 1: absolute CI impurity



6.0.2 Lecturer rpart method 2: robust absolute CI impurity



7 Forest and SHAP Objects

```
# min_relative_gain = c( 0.25, 0.30, 0.35, 0.40)
#minisplit = 500 #minibucket
```

```

p <- length(congo_predictors)
forest_grid <- ci_tree_control_grid(
  minsplit = 500L, # 1000
  minbucket = 100L, # 300
  minprob = 0.01,
  maxdepth = 15L,
  min_gain = 0.00001,
  min_relative_gain = c( 0.25, 0.30, 0.35), #0.3,0.35,
  mtry = c(15),
  ntree = c(50L, 100L)
)

```

```

# forest_tuning <- run_forest_tuning_batches(
#   formula = congo_ci_formula,
#   data = congo_model_dt,
#   forest_grid = forest_grid,
#   criterion_types = criterion_types,
#   tuning_metrics = ci_tuning_metrics,
#   tuning_selection_metric = "relative_validation_gain",
#   rank_name = "wealth",
#   outcome_name = "deadu5_num",
#   weights = congo_model_dt$sample_weight,
#   folds = runtime_params$tuning_folds,
#   workers = 4L,
#   progress_steps = min(20L, nrow(forest_grid)),
#   log_file = here("logs", "forest_tuning.log"),
#   seed = 20260508,
#   perturb = list(replace = FALSE, fraction = 0.7)
# )

# library(here)
# save(forest_tuning, file = here("data", "forest_tuning.rdata"))
load(here("data", "forest_tuning.rdata"))

```

```

#roc_auc
#relative_validation_gain
forest_best_by_type <- ci_select_best(
  forest_tuning,
  metric = "relative_validation_gain"
)

```

```

set.seed(20260509)
# forest_models_by_type <- setNames(
#   lapply(seq_len(nrow(forest_best_by_type)), function(i) {
#     setting <- forest_best_by_type[i]
#     ci_forest(
#       formula = congo_ci_formula,
#       data = congo_model_dt,
#       rank_name = "wealth",
#       outcome_name = "deadu5_num",
#       weights = congo_model_dt$sample_weight,
#       type = setting$type[1L],
#       control = ci_control_from_row(setting, include_mtry = FALSE),
#       ntree = as.integer(setting$ntree[1L]),
#       mtry = as.integer(setting$mtry[1L]),
#       perturb = list(replace = FALSE, fraction = 0.632)
#     )
#   })
# )

```

```
#   }),
#   forest_best_by_type$type
# )

# save(forest_models_by_type, file = here("data", "forest_models_by_type.rdata"))
load(here("data", "forest_models_by_type.rdata"))

selected_forest_fit <- forest_models_by_type[[selected_tree_type]]
forest_variable_importance <- collect_variable_importance(forest_models_by_type)
```

```
forest_complexity_metrics <- collect_complexity_metrics(forest_tuning)
forest_min_relative_gain_path <- min_relative_gain_path(
  forest_complexity_metrics,
  selected = forest_best_by_type
)
```

```
forest_selection_table <- ci_fit_summary_table(
  forest_tuning,
  selected = forest_best_by_type,
  metrics = unique(forest_tuning$summary$metric),
  include_percent = FALSE
)
forest_selection_table <- merge(
  forest_selection_table,
  forest_best_by_type[, .(type, ntree, mtry)],
  by = "type",
  all.x = TRUE,
  sort = FALSE
)
forest_selection_table[
  forest_best_by_type,
  `:=`(
    mean_terminal_nodes = i.mean_terminal_nodes,
    mean_max_depth = i.maxdepth
  ),
  on = "type"
]
forest_selection_table[, selection_basis := paste(
  runtime_params$tuning_folds,
  "fold cross-validation"
)]

forest_fit_wide <- copy(forest_selection_table)
forest_fit_table <- ci_collect_metrics(
  forest_tuning,
  selected = forest_best_by_type,
  metric = unique(forest_tuning$summary$metric),
  format = "tidy",
  include_train = TRUE
)
forest_best_controls <- forest_best_by_type[
  ,
  .(
    type, ntree, mtry, minsplitted, minbucket,
    minprob, maxdepth,
    min_gain, min_relative_gain
  )
]
```

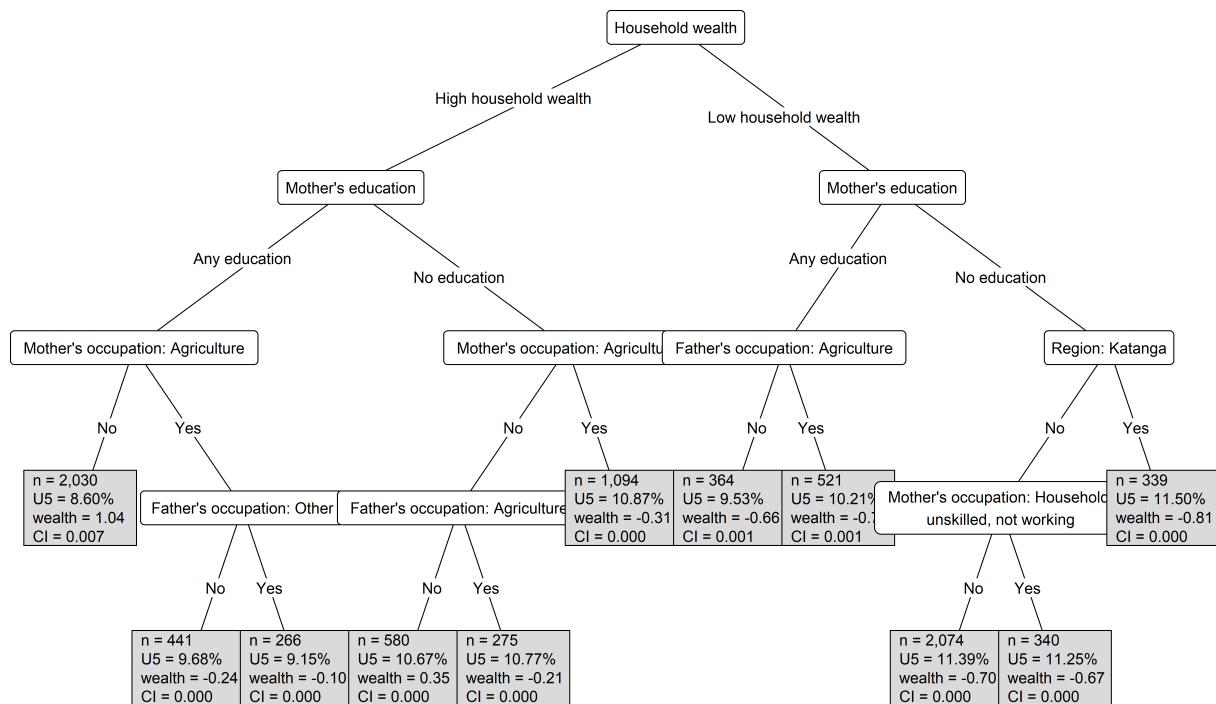
```
]
```

```
forest_surrogates_by_type <- setNames(
  lapply(seq_len(nrow(forest_best_by_type)), function(i) {
    #setting <- copy(forest_best_by_type[i])
    # use same tree settings
    setting <- copy(tree_best_by_type[type == forest_best_by_type$type[i]])
    type = setting$type[1L]

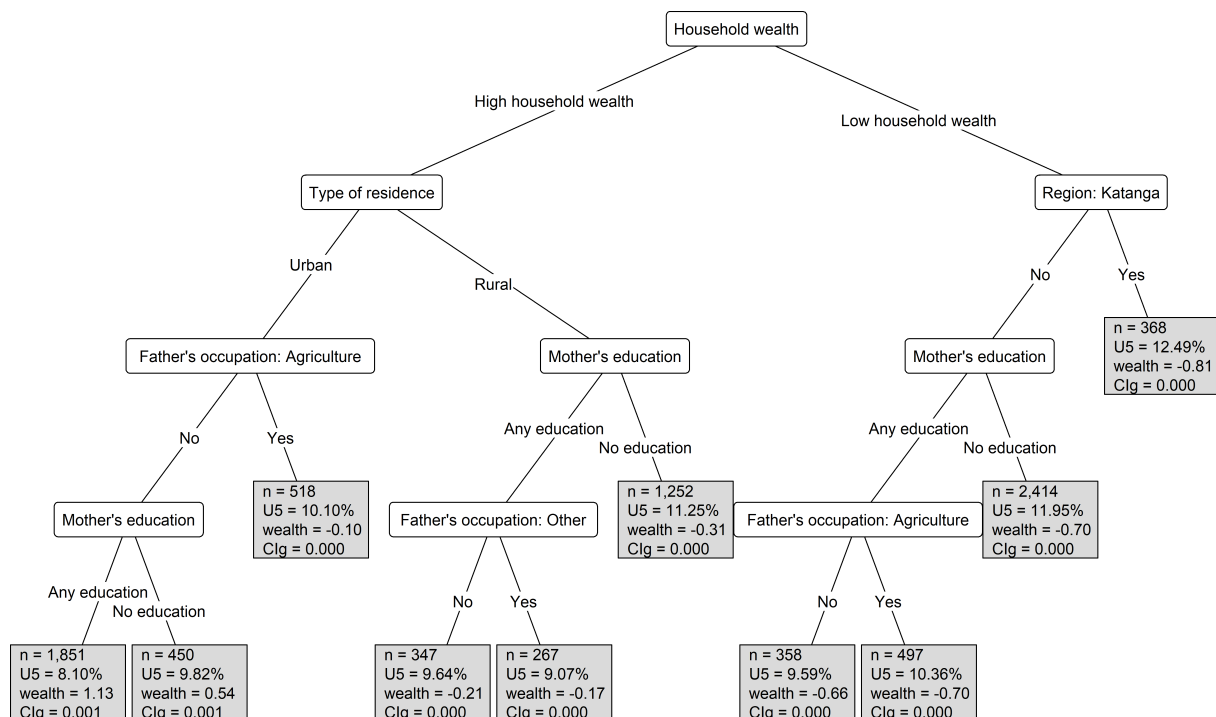
    if (type == "CIc"){
      setting$min_relative_gain <- 0.25
      #setting$minsplit <- 500L
      setting$minbucket <- 200L
    }else if (type == "L"){
      setting$min_relative_gain <- 0.3
      # setting$minsplit <- 1000L
      #setting$minbucket <- 200L
      setting$maxdepth <- 3L
    } else {
      setting$min_relative_gain <- 0.5
      setting$maxdepth <- 4L
      #setting$minsplit <- 1000L
      #setting$minbucket <- 200L
      #setting$min_gain <- 0.001
    }

    # # setting$minsplit <- 500L
    # # setting$minbucket <- 300L
    # # setting$maxdepth <- 6L
    ci_forest_surrogate(
      forest_fit = forest_models_by_type[[setting$type[1L]]],
      data = congo_model_dt,
      formula = congo_ci_formula,
      rank_name = "wealth",
      weights = congo_model_dt$sample_weight,
      type = setting$type[1L],
      control = ci_control_from_row(setting, include_mtry = FALSE),
      prediction_name = "forest_risk"
    )
  }),
  forest_best_by_type$type
)
selected_forest_surrogate <- forest_surrogates_by_type[[selected_tree_type]]
selected_forest_surrogate_summary <- as.data.table(
  ci_tree_terminal_summary(selected_forest_surrogate$fit)
)
```

```
ci_report_tree_plot(
  fit = forest_surrogates_by_type[["CI"]]$fit,
  data = forest_surrogates_by_type[["CI"]]$data,
  outcome_name = forest_surrogates_by_type[["CI"]]$prediction_name,
  outcome_label = "U5",
  ci_type = "CI",
  var_labels = congo_var_labels
)
```

```
ci_report_tree_plot(
  fit = forest_surrogates_by_type[["CIg"]] $\$$ fit,
  data = forest_surrogates_by_type[["CIg"]] $\$$ data,
  outcome_name = forest_surrogates_by_type[["CIg"]] $\$$ prediction_name,
  outcome_label = "U5",
  ci_type = "CIg",
  var_labels = congo_var_labels
)
```

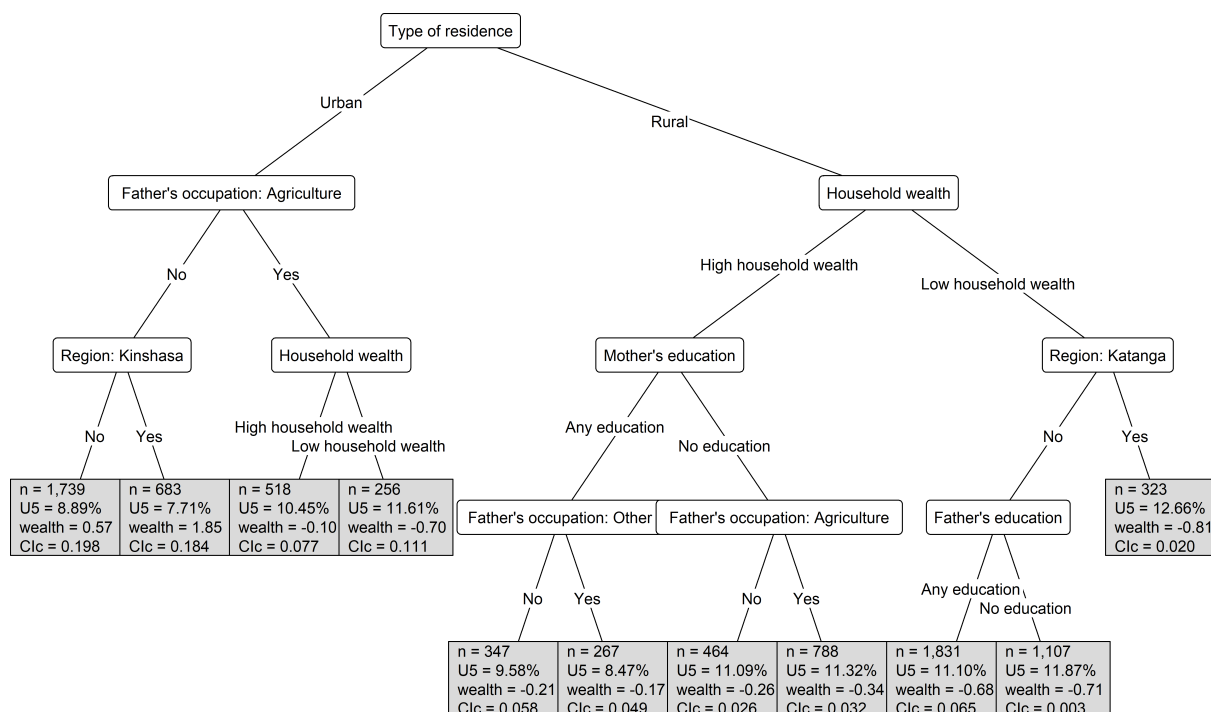


```
ci_report_tree_plot(
  fit = forest_surrogates_by_type[["CIc"]] $\$$ fit,
  data = forest_surrogates_by_type[["CIc"]] $\$$ data,
```

```

outcome_name = forest_surrogates_by_type[["CIc"]]$prediction_name,
outcome_label = "U5",
ci_type = "CIc",
var_labels = congo_var_labels
)

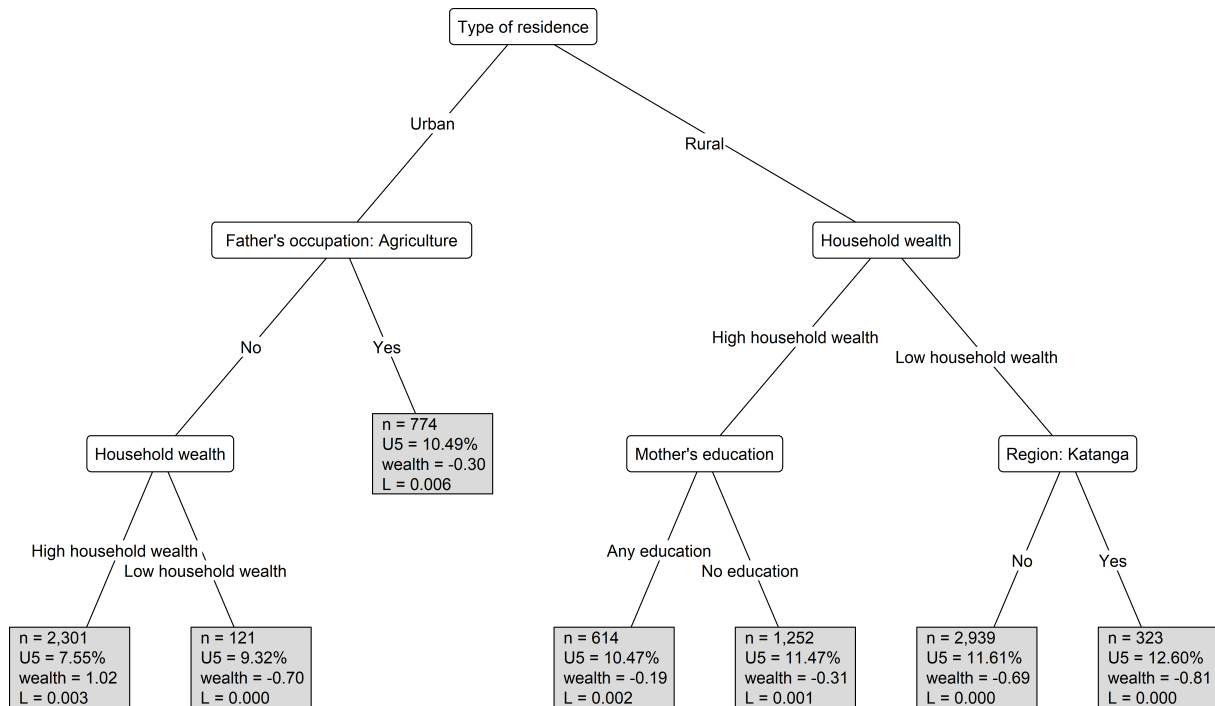
```



```

ci_report_tree_plot(
  fit = forest_surrogates_by_type[["L"]]$fit,
  data = forest_surrogates_by_type[["L"]]$data,
  outcome_name = forest_surrogates_by_type[["L"]]$prediction_name,
  outcome_label = "U5",
  ci_type = "L",
  var_labels = congo_var_labels
)

```



```

library(tidymodels)
ranger_analysis <- build_ranger_analysis_data(
  data = congo_model_dt,
  predictor_terms = congo_predictors,
  undersample = runtime_params$undersample,
  outcome_labels = outcome_labels
)
list2env(ranger_analysis, envir = environment())

```

<environment: R_GlobalEnv>

```
ranger_recipe <- recipe(deadu5_factor ~ ., data = ranger_fit_df)
```

```

ranger_model_spec <- rand_forest(
  mtry = tune(),
  min_n = tune(),
  trees = 500L
) |>
  set_mode("classification") |>
  set_engine("ranger", probability = TRUE, importance = "impurity")

```

```

ranger_resamples <- vfold_cv(
  ranger_fit_df,
  v = runtime_params$tuning_folds,
  strata = deadu5_factor
)

```

```

ranger_workflow <- workflow() |>
  add_model(ranger_model_spec) |>
  add_recipe(ranger_recipe) |>
  add_case_weights(sample_weight_case)

```

```

ranger_parameter_set <- hardhat::extract_parameter_set_dials(ranger_workflow) |>
  update(
    mtry = dials::mtry(range = c(10L, length(ranger_predictor_terms))),
    min_n = dials::min_n(range = c(200L, 800L))
  )
ranger_grid <- dials::grid_regular(
  ranger_parameter_set,
  levels = c(mtry = 4, min_n = 3)
)

```

```

ranger_metric_set <- yardstick::metric_set(
  yardstick::roc_auc,
  yardstick::accuracy,
  yardstick::sens,
  yardstick::spec,
  yardstick::f_meas
)
ranger_tune_results <- tune_grid(
  ranger_workflow,
  resamples = ranger_resamples,
  grid = ranger_grid,
  metrics = ranger_metric_set,
  control = control_grid(save_pred = TRUE)
)
ranger_tuning <- as.data.table(collect_metrics(ranger_tune_results))
ranger_cv_predictions <- as.data.table(collect_predictions(ranger_tune_results))

```

```

ranger_threshold_grid <- seq(0.05, 0.95, by = 0.01)

ranger_threshold_performance <- rbindlist(lapply(ranger_threshold_grid, function(cutoff) {
  threshold_dt <- copy(ranger_cv_predictions)

  threshold_dt[, pred_threshold := factor(
    fifelse(.pred_Died >= cutoff, "Died", "Survived"),
    levels = levels(deadu5_factor)
  )])

threshold_dt[
  ,
  .(
    threshold = cutoff,
    accuracy = mean(pred_threshold == deadu5_factor),
    sensitivity = yardstick::sens_vec(
      truth = deadu5_factor,
      estimate = pred_threshold,
      event_level = "first"
    ),
    specificity = yardstick::spec_vec(
      truth = deadu5_factor,
      estimate = pred_threshold,
      event_level = "first"
    ),
    f_meas = yardstick::f_meas_vec(
      truth = deadu5_factor,
      estimate = pred_threshold,
      event_level = "first"
    ),
  )
,

```

```

    predicted_died = mean(pred_threshold == "Died"),
    observed_died = mean(deadu5_factor == "Died")
  ),
  by = .config
]
}))
# knitr::kable(
#   ranger_threshold_performance,
#   caption = "Performance metrics across probability thresholds for the cross-validated ranger model"
# )

report_compact_table(
  ranger_tuning,
  digits = 4,
  caption = "Predictive ranger cross-validation summary"
)

```

Table 12: Predictive ranger cross-validation summary

mtry	min_n	.metric	.estimator	mean	n	std_err	.config
10	200	accuracy	binary	0.6108	10	0.0090	pre0_mod01_post0
10	200	f_meas	binary	0.3655	10	0.0155	pre0_mod01_post0
10	200	roc_auc	binary	0.6155	10	0.0111	pre0_mod01_post0
10	200	sens	binary	0.2774	10	0.0152	pre0_mod01_post0
10	200	spec	binary	0.8391	10	0.0148	pre0_mod01_post0
10	500	accuracy	binary	0.6080	10	0.0081	pre0_mod02_post0
10	500	f_meas	binary	0.2867	10	0.0200	pre0_mod02_post0
10	500	roc_auc	binary	0.6158	10	0.0097	pre0_mod02_post0
10	500	sens	binary	0.1960	10	0.0161	pre0_mod02_post0
10	500	spec	binary	0.8900	10	0.0102	pre0_mod02_post0
10	800	accuracy	binary	0.6072	10	0.0032	pre0_mod03_post0
10	800	f_meas	binary	0.1574	10	0.0167	pre0_mod03_post0
10	800	roc_auc	binary	0.6100	10	0.0101	pre0_mod03_post0
10	800	sens	binary	0.0917	10	0.0109	pre0_mod03_post0
10	800	spec	binary	0.9600	10	0.0056	pre0_mod03_post0
16	200	accuracy	binary	0.6066	10	0.0084	pre0_mod04_post0
16	200	f_meas	binary	0.3736	10	0.0146	pre0_mod04_post0
16	200	roc_auc	binary	0.6140	10	0.0104	pre0_mod04_post0
16	200	sens	binary	0.2900	10	0.0143	pre0_mod04_post0
16	200	spec	binary	0.8234	10	0.0131	pre0_mod04_post0
16	500	accuracy	binary	0.6104	10	0.0096	pre0_mod05_post0
16	500	f_meas	binary	0.3222	10	0.0176	pre0_mod05_post0
16	500	roc_auc	binary	0.6153	10	0.0102	pre0_mod05_post0
16	500	sens	binary	0.2293	10	0.0152	pre0_mod05_post0
16	500	spec	binary	0.8712	10	0.0131	pre0_mod05_post0
16	800	accuracy	binary	0.6076	10	0.0042	pre0_mod06_post0
16	800	f_meas	binary	0.2068	10	0.0144	pre0_mod06_post0
16	800	roc_auc	binary	0.6082	10	0.0102	pre0_mod06_post0
16	800	sens	binary	0.1273	10	0.0103	pre0_mod06_post0
16	800	spec	binary	0.9364	10	0.0088	pre0_mod06_post0
22	200	accuracy	binary	0.6080	10	0.0098	pre0_mod07_post0

22	200	f_meas	binary	0.3753	10	0.0185	pre0_mod07_post0
22	200	roc_auc	binary	0.6132	10	0.0105	pre0_mod07_post0
22	200	sens	binary	0.2923	10	0.0188	pre0_mod07_post0
22	200	spec	binary	0.8241	10	0.0151	pre0_mod07_post0
22	500	accuracy	binary	0.6118	10	0.0083	pre0_mod08_post0
22	500	f_meas	binary	0.3459	10	0.0158	pre0_mod08_post0
22	500	roc_auc	binary	0.6146	10	0.0100	pre0_mod08_post0
22	500	sens	binary	0.2546	10	0.0159	pre0_mod08_post0
22	500	spec	binary	0.8563	10	0.0142	pre0_mod08_post0
22	800	accuracy	binary	0.6071	10	0.0051	pre0_mod09_post0
22	800	f_meas	binary	0.2266	10	0.0194	pre0_mod09_post0
22	800	roc_auc	binary	0.6082	10	0.0104	pre0_mod09_post0
22	800	sens	binary	0.1445	10	0.0149	pre0_mod09_post0
22	800	spec	binary	0.9238	10	0.0115	pre0_mod09_post0
29	200	accuracy	binary	0.6090	10	0.0106	pre0_mod10_post0
29	200	f_meas	binary	0.3888	10	0.0175	pre0_mod10_post0
29	200	roc_auc	binary	0.6126	10	0.0103	pre0_mod10_post0
29	200	sens	binary	0.3072	10	0.0162	pre0_mod10_post0
29	200	spec	binary	0.8155	10	0.0130	pre0_mod10_post0
29	500	accuracy	binary	0.6053	10	0.0087	pre0_mod11_post0
29	500	f_meas	binary	0.3425	10	0.0152	pre0_mod11_post0
29	500	roc_auc	binary	0.6141	10	0.0099	pre0_mod11_post0
29	500	sens	binary	0.2546	10	0.0149	pre0_mod11_post0
29	500	spec	binary	0.8453	10	0.0147	pre0_mod11_post0
29	800	accuracy	binary	0.6076	10	0.0050	pre0_mod12_post0
29	800	f_meas	binary	0.2367	10	0.0157	pre0_mod12_post0
29	800	roc_auc	binary	0.6084	10	0.0100	pre0_mod12_post0
29	800	sens	binary	0.1514	10	0.0123	pre0_mod12_post0
29	800	spec	binary	0.9199	10	0.0089	pre0_mod12_post0

```
selected_ranger_setting <- select_best(ranger_tune_results, metric = "f_meas")

selected_ranger_threshold <- ranger_threshold_performance[
  .config == selected_ranger_setting$.config
][
  order(-f_meas, -sensitivity, -specificity, threshold)
][1L]
```

```
report_compact_table(
  as.data.table(as.data.frame(selected_ranger_setting)),
  digits = 4,
  caption = "Selected predictive ranger setting"
)
```

Table 13: Selected predictive ranger setting

mtry	min_n	.config
29	200	pre0_mod10_post0

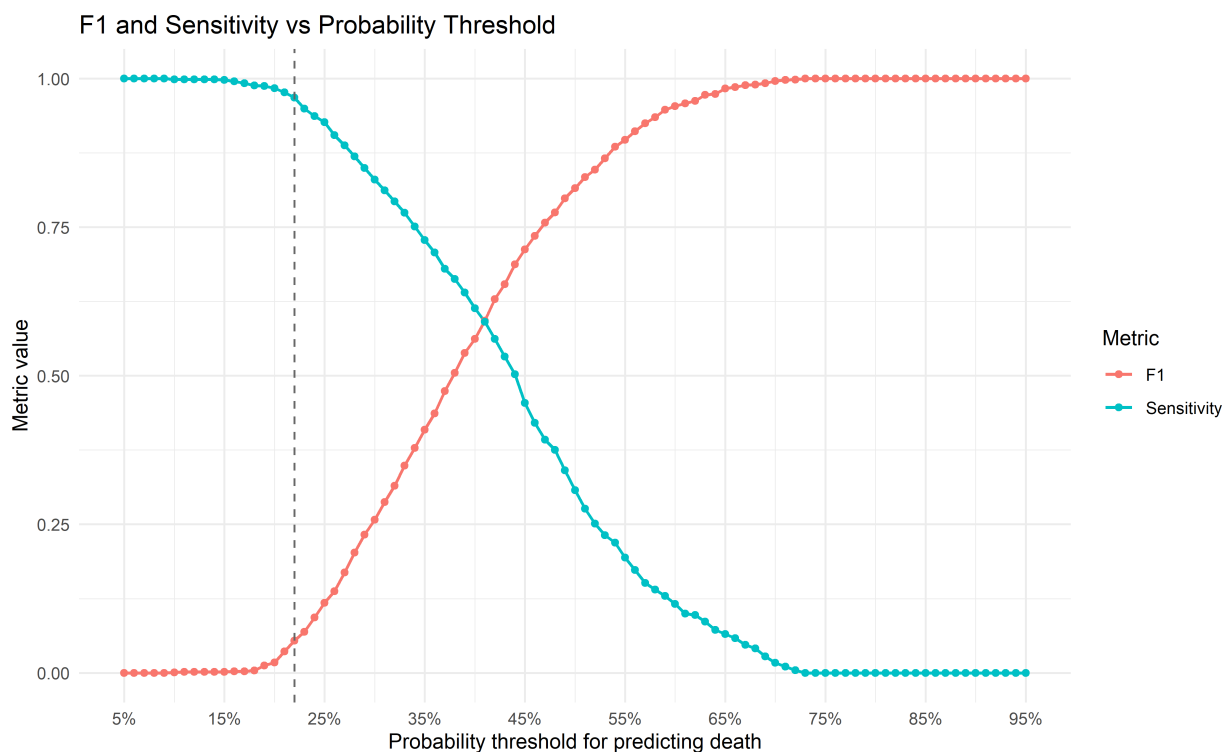
```

plot_threshold_dt <- ranger_threshold_performance[
  .config == selected_ranger_setting$.config,
  .(threshold, F1 = specificity, Sensitivity = sensitivity)
]

plot_threshold_long <- melt(
  plot_threshold_dt,
  id.vars = "threshold",
  variable.name = "Metric",
  value.name = "value"
)

ggplot(plot_threshold_long, aes(x = threshold, y = value, colour = Metric)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1.4) +
  geom_vline(
    xintercept = selected_ranger_threshold$threshold,
    linetype = "dashed",
    colour = "grey40"
  ) +
  scale_x_continuous(
    breaks = seq(0.05, 0.95, by = 0.10),
    labels = scales::percent_format(accuracy = 1)
  ) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(
    title = "F1 and Sensitivity vs Probability Threshold",
    x = "Probability threshold for predicting death",
    y = "Metric value",
    colour = "Metric"
  ) +
  theme_minimal()

```



```
report_compact_table(
  selected_ranger_threshold,
  digits = 4,
  caption = "Selected predictive ranger probability threshold"
)
```

Table 14: Selected predictive ranger probability threshold

.config	threshold	accuracy	sensitivity	specificity	f_meas	predicted_died	ob
pre0_mod10_post0	0.22	0.4254	0.9679	0.0542	0.5779	0.9548	

```
final_ranger_workflow <- finalize_workflow(ranger_workflow, selected_ranger_setting)
predictive_forest_fit <- workflows::fit(final_ranger_workflow, data = ranger_fit_df)
```

```
predict_ranger_risk <- function(object, newdata) {
  predict(object, new_data = as.data.frame(newdata), type = "prob")$.pred_Died
}

predict_ranger_class <- function(
  object,
  newdata,
  threshold = selected_ranger_threshold$threshold
) {
  factor(
    ifelse(predict_ranger_risk(object, newdata) >= threshold, "Died", "Survived"),
    levels = c("Died", "Survived")
  )
}
```

```
shap_rows <- sort(sample.int(nrow(ranger_fit_df), max(shap_n, nrow(ranger_fit_df))))
shap_x_background <- ranger_fit_df[, ranger_predictor_terms, drop = FALSE]
shap_x_eval <- ranger_fit_df[shap_rows, ranger_predictor_terms, drop = FALSE]
shap_prediction <- as.numeric(predict_ranger_risk(predictive_forest_fit, shap_x_eval))
```

```
selected_forest_shap <- fastshap::explain(
  object = predictive_forest_fit,
  X = shap_x_background,
  pred_wrapper = predict_ranger_risk,
  newdata = shap_x_eval,
  nsim = shap_nsim,
  adjust = shap_nsim > 1L
)
colnames(selected_forest_shap) <- canonicalize_model_terms(
  colnames(selected_forest_shap),
  congo_predictors
)

shap_feature_values <- copy(as.data.table(shap_x_eval))
shap_feature_values[, observation_id := seq_len(.N)]
selected_forest_shap_dt <- as.data.table(selected_forest_shap)
selected_forest_shap_dt[, observation_id := seq_len(.N)]

shap_summary_long <- melt(
  selected_forest_shap_dt,
  id.vars = "observation_id",
```



```

    variable.name = "feature",
    value.name = "shap_value",
    variable.factor = FALSE
  )
shap_value_long <- melt(
  shap_feature_values,
  id.vars = "observation_id",
  variable.name = "feature",
  value.name = "feature_value",
  variable.factor = FALSE
)
shap_summary_long <- merge(
  shap_summary_long,
  shap_value_long,
  by = c("observation_id", "feature"),
  all.x = TRUE,
  sort = FALSE
)
shap_summary_long[
  ,
  feature_value_numeric := suppressWarnings(as.numeric(as.character(feature_value)))
]
shap_summary_long[
  is.na(feature_value_numeric),
  feature_value_numeric := as.numeric(factor(feature_value)) - 1
]
shap_summary_long[
  ,
  feature_value_scaled := {
    range_value <- range(feature_value_numeric, na.rm = TRUE)
    fifelse(
      rep(all(is.finite(range_value)) && diff(range_value) > .Machine$double.eps, .N),
      (feature_value_numeric - range_value[1L]) / diff(range_value),
      0.5
    )
  },
  by = feature
]
shap_summary_long[, feature_label := format_variable_label(feature)]
shap_summary_long[
  ,
  mean_abs_shap := mean(abs(shap_value), na.rm = TRUE),
  by = feature
]
setcolororder(
  shap_summary_long,
  c(
    "feature",
    "feature_label",
    "observation_id",
    "shap_value",
    "feature_value",
    "feature_value_numeric",
    "feature_value_scaled",
    "mean_abs_shap"
  )
)

```

```
shap_ci_decomp <- shap_conc_decomp(
  shap = selected_forest_shap,
  rank = ranger_dt$wealth[shap_rows],
  type = "CI",
  prediction = shap_prediction,
  weights = ranger_dt$sample_weight[shap_rows]
)
```

```
shap_ci_diag <- as.data.table(shap_ci_decomp$diagnostics)
shap_ci_contrib <- as.data.table(shap_ci_decomp$contributions)[order(pct_contribution)]
report_compact_table(
  shap_ci_diag,
  digits = 4,
  caption = "SHAP decomposition diagnostics: CI"
)
```

Table 15: SHAP decomposition diagnostics: CI

n	weight_sum	type	mean_prediction	concentration_index	signed_concentration_index
2146	2148.538	CI	0.4001	0.0816	-0.0816

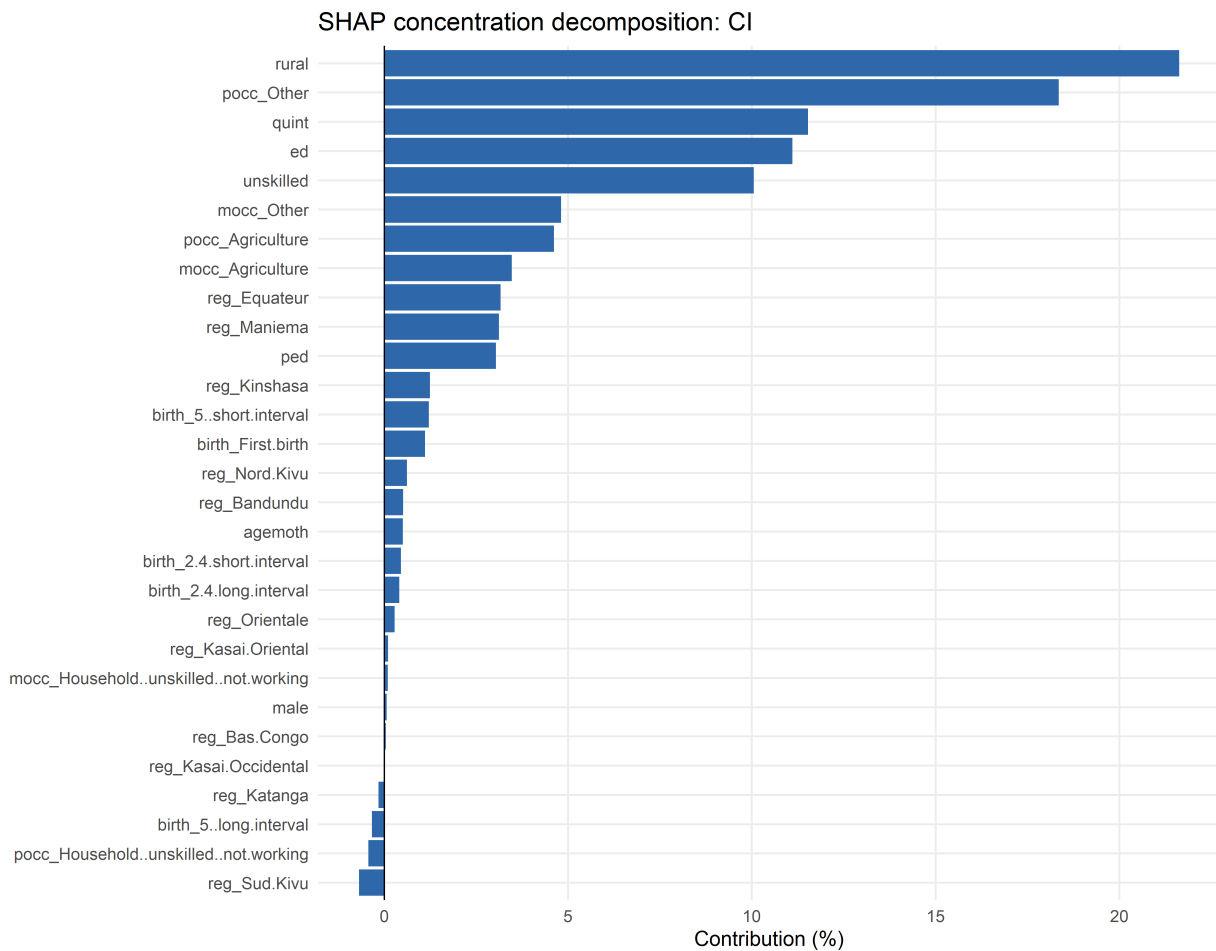
```
report_compact_table(
  shap_ci_contrib,
  digits = 4,
  caption = "SHAP decomposition contributions: CI"
)
```

Table 16: SHAP decomposition contributions: CI

feature	D_k_SHAP	pct_contribution	abs_contribution
reg_Sud.Kivu	-0.0006	-0.6880	0.0006
pocc_Household..unskilled..not.working	-0.0004	-0.4304	0.0004
birth_5..long.interval	-0.0003	-0.3327	0.0003
reg_Katanga	-0.0001	-0.1553	0.0001
reg_Kasai.Occidental	0.0000	0.0066	0.0000
reg_Bas.Congo	0.0000	0.0481	0.0000
male	0.0001	0.0682	0.0001
mocc_Household..unskilled..not.working	0.0001	0.0941	0.0001
reg_Kasai.Oriental	0.0001	0.1055	0.0001
reg_Orientale	0.0002	0.2832	0.0002
birth_2.4.long.interval	0.0003	0.4086	0.0003
birth_2.4.short.interval	0.0004	0.4576	0.0004
agemoth	0.0004	0.5046	0.0004
reg_Bandundu	0.0004	0.5173	0.0004
reg_Nord.Kivu	0.0005	0.6190	0.0005
birth_First.birth	0.0009	1.1123	0.0009
birth_5..short.interval	0.0010	1.2144	0.0010
reg_Kinshasa	0.0010	1.2459	0.0010
ped	0.0025	3.0426	0.0025
reg_Maniema	0.0025	3.1242	0.0025
reg_Equateur	0.0026	3.1651	0.0026

mocc_Agriculture	0.0028	3.4735	0.0028
pocc_Agriculture	0.0038	4.6191	0.0038
mocc_Other	0.0039	4.8135	0.0039
unskilled	0.0082	10.0577	0.0082
ed	0.0091	11.1101	0.0091
quint	0.0094	11.5305	0.0094
pocc_Other	0.0150	18.3544	0.0150
rural	0.0177	21.6302	0.0177

```
ggplot(shap_ci_contrib, aes(x = pct_contribution, y = reorder(feature, pct_contribution))) +
  geom_col(fill = "#3268A8") +
  geom_vline(xintercept = 0, color = "black", linewidth = 0.4) +
  labs(x = "Contribution (%)", y = NULL, title = "SHAP concentration decomposition: CI") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```



```
shap_cig_decomp <- shap_conc_decomp(
  shap = selected_forest_shap,
  rank = ranger_dt$wealth[shap_rows],
  type = "CIg",
  prediction = shap_prediction,
  weights = ranger_dt$sample_weight[shap_rows]
)
```

```
shap_cig_diag <- as.data.table(shap_cig_decomp$diagnostics)
shap_cig_contrib <- as.data.table(shap_cig_decomp$contributions)[order(pct_contribution)]
report_compact_table(
  shap_cig_diag,
  digits = 4,
  caption = "SHAP decomposition diagnostics: CIG"
)
```

Table 17: SHAP decomposition diagnostics: CIG

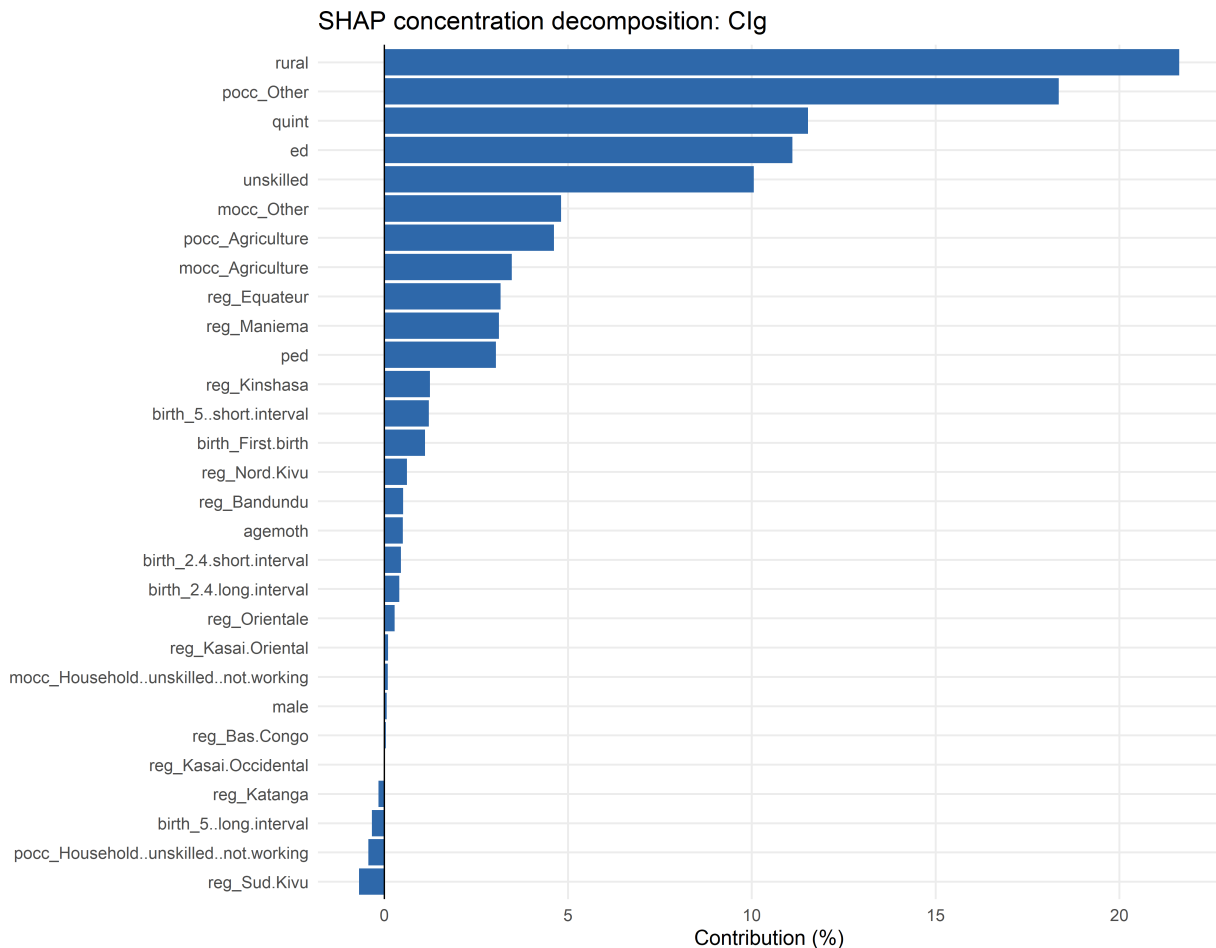
n	weight_sum	type	mean_prediction	concentration_index	signed_concentration_index
2146	2148.538	CIG	0.4001	0.0327	-0.0327

```
report_compact_table(
  shap_cig_contrib,
  digits = 4,
  caption = "SHAP decomposition contributions: CIG"
)
```

Table 18: SHAP decomposition contributions: CIG

feature	D_k_SHAP	pct_contribution	abs_contribution
reg_Sud.Kivu	-0.0002	-0.6880	0.0002
pocc_Household..unskilled..not.working	-0.0001	-0.4304	0.0001
birth_5..long.interval	-0.0001	-0.3327	0.0001
reg_Katanga	-0.0001	-0.1553	0.0001
reg_Kasai.Occidental	0.0000	0.0066	0.0000
reg_Bas.Congo	0.0000	0.0481	0.0000
male	0.0000	0.0682	0.0000
mocc_Household..unskilled..not.working	0.0000	0.0941	0.0000
reg_Kasai.Oriental	0.0000	0.1055	0.0000
reg_Orientale	0.0001	0.2832	0.0001
birth_2.4.long.interval	0.0001	0.4086	0.0001
birth_2.4.short.interval	0.0001	0.4576	0.0001
agemoth	0.0002	0.5046	0.0002
reg_Bandundu	0.0002	0.5173	0.0002
reg_Nord.Kivu	0.0002	0.6190	0.0002
birth_First.birth	0.0004	1.1123	0.0004
birth_5..short.interval	0.0004	1.2144	0.0004
reg_Kinshasa	0.0004	1.2459	0.0004
ped	0.0010	3.0426	0.0010
reg_Maniema	0.0010	3.1242	0.0010
reg_Equateur	0.0010	3.1651	0.0010
mocc_Agriculture	0.0011	3.4735	0.0011
pocc_Agriculture	0.0015	4.6191	0.0015
mocc_Other	0.0016	4.8135	0.0016
unskilled	0.0033	10.0577	0.0033
ed	0.0036	11.1101	0.0036
quint	0.0038	11.5305	0.0038
pocc_Other	0.0060	18.3544	0.0060

```
ggplot(shap_cig_contrib, aes(x = pct_contribution, y = reorder(feature, pct_contribution))) +
  geom_col(fill = "#3268A8") +
  geom_vline(xintercept = 0, color = "black", linewidth = 0.4) +
  labs(x = "Contribution (%)", y = NULL, title = "SHAP concentration decomposition: CIG") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```



```
shap_cic_decomp <- shap_conc_decomp(
  shap = selected_forest_shap,
  rank = ranger_dt$wealth[shap_rows],
  type = "CIG",
  prediction = shap_prediction,
  weights = ranger_dt$sample_weight[shap_rows]
)
```

```
shap_cic_diag <- as.data.table(shap_cic_decomp$diagnostics)
shap_cic_contrib <- as.data.table(shap_cic_decomp$contributions)[order(pct_contribution)]
report_compact_table(
  shap_cic_diag,
  digits = 4,
  caption = "SHAP decomposition diagnostics: CIG"
)
```

Table 19: SHAP decomposition

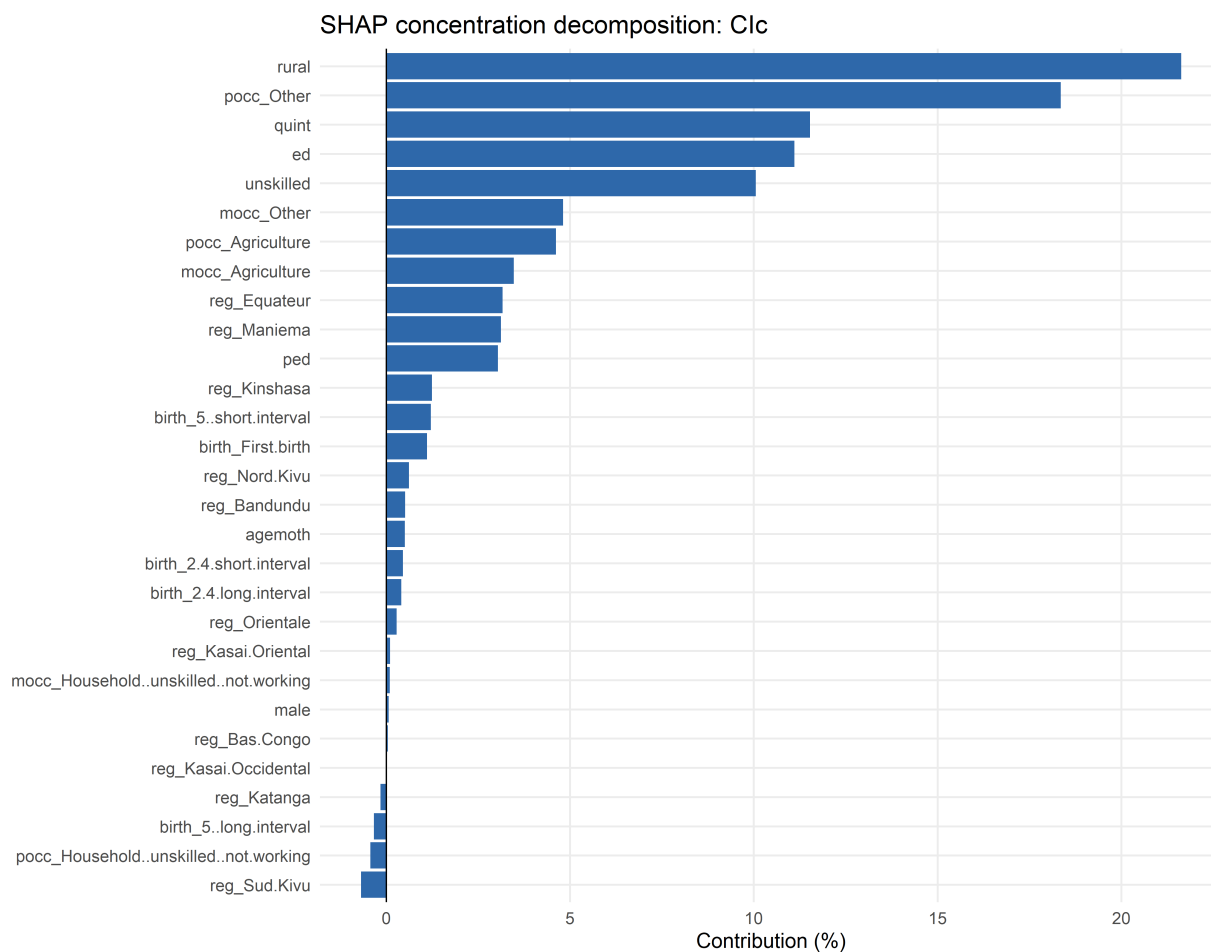
n	weight_sum	type	mean_prediction	concentration_index	signed_concentration_index
2146	2148.538	CIc	0.4001	0.2268	-0.2268

```
report_compact_table(
  shap_cic_contrib,
  digits = 4,
  caption = "SHAP decomposition contributions: CIc"
)
```

Table 20: SHAP decomposition contributions: CIc

feature	D_k_SHAP	pct_contribution	abs_contribution
reg_Sud.Kivu	-0.0016	-0.6880	0.0016
pocc_Household..unskilled..not.working	-0.0010	-0.4304	0.0010
birth_5..long.interval	-0.0008	-0.3327	0.0008
reg_Katanga	-0.0004	-0.1553	0.0004
reg_Kasai.Occidental	0.0000	0.0066	0.0000
reg_Bas.Congo	0.0001	0.0481	0.0001
male	0.0002	0.0682	0.0002
mocc_Household..unskilled..not.working	0.0002	0.0941	0.0002
reg_Kasai.Oriental	0.0002	0.1055	0.0002
reg_Orientale	0.0006	0.2832	0.0006
birth_2.4.long.interval	0.0009	0.4086	0.0009
birth_2.4.short.interval	0.0010	0.4576	0.0010
agemoth	0.0011	0.5046	0.0011
reg_Bandundu	0.0012	0.5173	0.0012
reg_Nord.Kivu	0.0014	0.6190	0.0014
birth_First.birth	0.0025	1.1123	0.0025
birth_5..short.interval	0.0028	1.2144	0.0028
reg_Kinshasa	0.0028	1.2459	0.0028
ped	0.0069	3.0426	0.0069
reg_Maniema	0.0071	3.1242	0.0071
reg_Equateur	0.0072	3.1651	0.0072
mocc_Agriculture	0.0079	3.4735	0.0079
pocc_Agriculture	0.0105	4.6191	0.0105
mocc_Other	0.0109	4.8135	0.0109
unskilled	0.0228	10.0577	0.0228
ed	0.0252	11.1101	0.0252
quint	0.0262	11.5305	0.0262
pocc_Other	0.0416	18.3544	0.0416
rural	0.0491	21.6302	0.0491

```
ggplot(shap_cic_contrib, aes(x = pct_contribution, y = reorder(feature, pct_contribution))) +
  geom_col(fill = "#3268A8") +
  geom_vline(xintercept = 0, color = "black", linewidth = 0.4) +
  labs(x = "Contribution (%)", y = NULL, title = "SHAP concentration decomposition: CIc") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```



```
shap_l_decomp <- shap_conc_decomp(
  shap = selected_forest_shap,
  rank = ranger_dt$wealth[shap_rows],
  type = "L",
  prediction = shap_prediction,
  weights = ranger_dt$sample_weight[shap_rows]
)
```

```
shap_l_diag <- as.data.table(shap_l_decomp$diagnostics)
shap_l_contrib <- as.data.table(shap_l_decomp$contributions)[order(pct_contribution)]
report_compact_table(
  shap_l_diag,
  digits = 4,
  caption = "SHAP decomposition diagnostics: L"
)
```

Table 21: SHAP decomposition

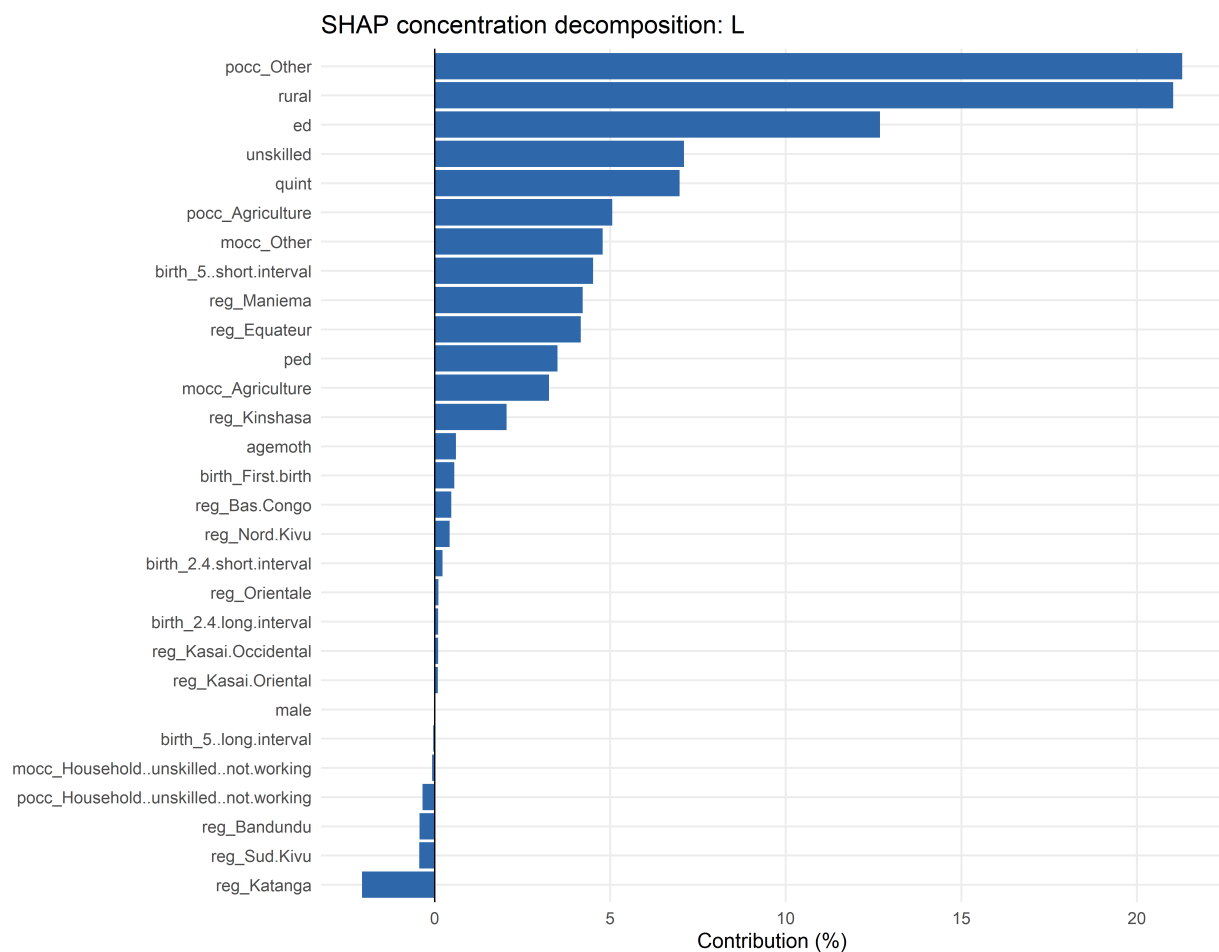
n	weight_sum	type	mean_prediction	concentration_index	signed_concentration_index
2146	2148.538	L	0.4001	0.3252	0.3252

```
report_compact_table(
  shap_l_contrib,
  digits = 4,
  caption = "SHAP decomposition contributions: L"
)
```

Table 22: SHAP decomposition contributions: L

feature	D_k_SHAP	pct_contribution	abs_contribution
reg_Katanga	-0.0067	-2.0676	0.0067
reg_Sud.Kivu	-0.0014	-0.4336	0.0014
reg_Bandundu	-0.0014	-0.4303	0.0014
pocc_Household..unskilled..not.working	-0.0011	-0.3431	0.0011
mocc_Household..unskilled..not.working	-0.0002	-0.0683	0.0002
birth_5..long.interval	-0.0001	-0.0399	0.0001
male	0.0000	-0.0047	0.0000
reg_Kasai.Oriental	0.0003	0.0963	0.0003
reg_Kasai.Occidental	0.0003	0.1018	0.0003
birth_2.4.long.interval	0.0003	0.1059	0.0003
reg_Orientale	0.0003	0.1075	0.0003
birth_2.4.short.interval	0.0008	0.2307	0.0008
reg_Nord.Kivu	0.0014	0.4292	0.0014
reg_Bas.Congo	0.0015	0.4738	0.0015
birth_First.birth	0.0018	0.5624	0.0018
agemoth	0.0020	0.6129	0.0020
reg_Kinshasa	0.0067	2.0537	0.0067
mocc_Agriculture	0.0106	3.2623	0.0106
ped	0.0114	3.5001	0.0114
reg_Equateur	0.0135	4.1651	0.0135
reg_Maniema	0.0137	4.2184	0.0137
birth_5..short.interval	0.0147	4.5178	0.0147
mocc_Other	0.0156	4.7889	0.0156
pocc_Agriculture	0.0165	5.0600	0.0165
quint	0.0227	6.9797	0.0227
unskilled	0.0231	7.1068	0.0231
ed	0.0413	12.6836	0.0413
rural	0.0684	21.0384	0.0684
pocc_Other	0.0692	21.2922	0.0692

```
ggplot(shap_l_contrib, aes(x = pct_contribution, y = reorder(feature, pct_contribution))) +
  geom_col(fill = "#3268A8") +
  geom_vline(xintercept = 0, color = "black", linewidth = 0.4) +
  labs(x = "Contribution (%)", y = NULL, title = "SHAP concentration decomposition: L") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```

```
selected_shap_contributions <- rbindlist(list(
  as.data.table(shap_ci_decomp$contributions)[, criterion := "CI"],
  as.data.table(shap_cig_decomp$contributions)[, criterion := "CIg"],
  as.data.table(shap_cic_decomp$contributions)[, criterion := "CIc"],
  as.data.table(shap_l_decomp$contributions)[, criterion := "L"]
), fill = TRUE)
selected_shap_contributions[, feature := canonicalize_model_terms(feature, congo_predictors)]
setcolororder(selected_shap_contributions, c("criterion", setdiff(names(selected_shap_contributions), "criterion")))

selected_shap_diagnostics <- rbindlist(list(
  as.data.table(shap_ci_decomp$diagnostics)[, criterion := "CI"],
  as.data.table(shap_cig_decomp$diagnostics)[, criterion := "CIg"],
  as.data.table(shap_cic_decomp$diagnostics)[, criterion := "CIc"],
  as.data.table(shap_l_decomp$diagnostics)[, criterion := "L"]
), fill = TRUE)
setcolororder(selected_shap_diagnostics, c("criterion", setdiff(names(selected_shap_diagnostics), "criterion")))

forest_importance_out <- copy(forest_variable_importance)
forest_importance_out[, variable_label := fifelse(
  variable %in% names(congo_var_labels),
  congo_var_labels[variable],
  variable
)]

report_compact_table(
  forest_importance_out,
  digits = 4,
  caption = "Forest variable importance preview"
```

Table 23: Forest variable importance preview

critereon	variable	importance	rank	variable_label
CI	ed	0.0225	1	Mother's education
CI	quint	0.0102	2	Household wealth
CI	mocc_Agriculture	0.0072	3	Mother's occupation: Agriculture
CI	pocc_Agriculture	0.0066	4	Father's occupation: Agriculture
CI	pocc_Other	0.0023	5	Father's occupation: Other
CI	reg_Kinshasa	0.0015	6	Region: Kinshasa
CI	reg_Kasai.Occidental	0.0007	7	Region: Kasai Occidental
CI	unskilled	0.0007	8	Skilled birth attendance
CI	reg_Katanga	0.0004	9	Region: Katanga
CI	birth_First.birth	0.0004	10	Birth order and interval: First b
CI	reg_Equateur	0.0003	11	Region: Equateur
CI	birth_5..short.interval	0.0003	12	Birth order and interval: 5+ sho
CIc	ed	0.0120	1	Mother's education
CIc	quint	0.0051	2	Household wealth
CIc	pocc_Agriculture	0.0042	3	Father's occupation: Agriculture
CIc	reg_Kinshasa	0.0027	4	Region: Kinshasa
CIc	mocc_Agriculture	0.0020	5	Mother's occupation: Agriculture
CIc	pocc_Other	0.0017	6	Father's occupation: Other
CIc	rural	0.0015	7	Type of residence
CIc	reg_Katanga	0.0012	8	Region: Katanga
CIc	mocc_Other	0.0004	9	Mother's occupation: Other
CIc	unskilled	0.0003	10	Skilled birth attendance
CIc	reg_Equateur	0.0002	11	Region: Equateur
CIc	mocc_Household..unskilled..not.working	0.0002	12	Mother's occupation: Household
CIg	ed	0.0023	1	Mother's education
CIg	rural	0.0014	2	Type of residence
CIg	quint	0.0014	3	Household wealth
CIg	pocc_Agriculture	0.0007	4	Father's occupation: Agriculture
CIg	mocc_Agriculture	0.0006	5	Mother's occupation: Agriculture
CIg	pocc_Other	0.0005	6	Father's occupation: Other
CIg	reg_Katanga	0.0002	7	Region: Katanga
CIg	reg_Kinshasa	0.0001	8	Region: Kinshasa
CIg	pocc_Household..unskilled..not.working	0.0001	9	Father's occupation: Household
CIg	unskilled	0.0001	10	Skilled birth attendance
CIg	mocc_Other	0.0001	11	Mother's occupation: Other
CIg	reg_Orientale	0.0000	12	Region: Orientale
L	rural	0.1085	1	Type of residence
L	mocc_Agriculture	0.0333	2	Mother's occupation: Agriculture
L	ed	0.0249	3	Mother's education
L	quint	0.0146	4	Household wealth
L	pocc_Agriculture	0.0067	5	Father's occupation: Agriculture
L	pocc_Other	0.0046	6	Father's occupation: Other
L	reg_Kinshasa	0.0014	7	Region: Kinshasa
L	reg_Equateur	0.0003	8	Region: Equateur

L	reg_Katanga	0.0002	9	Region: Katanga
L	reg_Bandundu	0.0001	10	Region: Bandundu
L	mocc_Other	0.0000	11	Mother's occupation: Other
L	male	0.0000	12	Sex of the child

8 Compare Methods

Once the regression, tree, forest, and SHAP objects exist, the next step is to rank variables and assemble the comparison tables used in the main report.

```
linear_rank <- classical_decomposition_table[
  criterion == "CI"
][
  order(-abs(regression_pct_contribution)),
  .(variable, method = "Linear decomposition", rank = seq_len(.N))
]
tree_rank <- rank_from_named_vector(selected_tree_fit$variable.importance, "Selected CI-tree")
forest_rank <- rank_from_named_vector(selected_forest_fit$variable.importance, "Selected CI-forest")
shap_rank <- selected_shap_contributions[
  criterion == "CI"
][
  order(-abs_contribution),
  .(variable = feature, method = "Selected forest SHAP", rank = seq_len(.N))
]
importance_long <- rbindlist(list(linear_rank, tree_rank, forest_rank, shap_rank), fill = TRUE)

selected_variables <- classical_decomposition_table[
  criterion == "CI"
][
  order(-abs(regression_pct_contribution))
]$variable[seq_len(max(15L, classical_decomposition_table[criterion == "CI", .N]))]
selected_variables <- selected_variables[!is.na(selected_variables) & nzchar(selected_variables)]

method_comparison_table <- data.table(variable = selected_variables)
method_comparison_table[, variable_label := format_variable_label(variable)]
regression_wide <- dcast(
  classical_decomposition_table[
    criterion %in% c("CI", "CIg", "CIc"),
    .(variable, criterion, regression_pct_contribution)
  ],
  variable ~ criterion,
  value.var = "regression_pct_contribution"
)
method_comparison_table <- merge(
  method_comparison_table,
  regression_wide[
    ,
    .(
      variable,
      regression_ci_pct_contribution = CI,
      regression_cig_pct_contribution = CIg,
      regression_cic_pct_contribution = CIc
    )
  ],
  by = "variable",
  all.x = TRUE
)
```

```

)
shap_wide <- dcast(
  selected_shap_contributions[
    criterion %in% criterion_types,
    .(variable = feature, criterion, pct_contribution)
  ],
  variable ~ criterion,
  value.var = "pct_contribution"
)
shap_wide <- shap_wide[
  ,
  .(
    variable,
    forest_shap_ci_pct_contribution = CI,
    forest_shap_cig_pct_contribution = CIg,
    forest_shap_cic_pct_contribution = CIc,
    forest_shap_l_pct_contribution = L
  )
]
method_comparison_table <- merge(
  method_comparison_table,
  shap_wide,
  by = "variable",
  all.x = TRUE
)
method_comparison_table <- method_comparison_table[
  order(-abs(regression_ci_pct_contribution), -abs(forest_shap_ci_pct_contribution))
]

importance_wide <- dcast(
  importance_long,
  rank ~ method,
  value.var = "variable",
  fun.aggregate = function(x) paste(unique(x), collapse = ", ")
)
for (col in setdiff(names(importance_wide), "rank")) {
  importance_wide[, (col) := format_variable_label(get(col))]
}
importance_wide <- importance_wide[rank <= 10L]

report_compact_table(
  method_comparison_table,
  digits = 3,
  caption = "Selected percentage-contribution comparison table"
)

```

variable	variable_label	regression
ed	Mother's education	
pocc_Agriculture	Father's occupation: Agriculture	
mocc_Agriculture	Mother's occupation: Agriculture	
quint	Household wealth	
rural	Type of residence	
unskilled	Skilled birth attendance	

reg_Equateur	Region: Equateur
pocc_Household..unskilled..not.working	Father's occupation: Household, unskilled, not working
reg_Kinshasa	Region: Kinshasa
reg_Sud.Kivu	Region: Sud-Kivu
reg_Kasai.Oriental	Region: Kasai Oriental
ped	Father's education
reg_Maniema	Region: Maniema
reg_Orientale	Region: Orientale
reg_Bas.Congo	Region: Bas-Congo
mocc_Household..unskilled..not.working	Mother's occupation: Household, unskilled, not working
birth_5..short.interval	Birth order and interval: 5+ short interval
reg_Kasai.Occidental	Region: Kasai Occidental
birth_5..long.interval	Birth order and interval: 5+ long interval
male	Sex of the child
reg_Katanga	Region: Katanga
agemoth	Mother's age at birth
reg_Nord.Kivu	Region: Nord-Kivu
birth_2.4.long.interval	Birth order and interval: 2-4 long interval
birth_2.4.short.interval	Birth order and interval: 2-4 short interval

```
report_compact_table(
  importance_wide,
  caption = "Ranked variables across decomposition and tree-based methods"
)
```

Table 25: Ranked variables across decomposition and tree-based

rank	Linear decomposition	Selected CI-forest	Selected
1	Mother's education	Type of residence	Type of r
2	Father's occupation: Agriculture	Mother's occupation: Agriculture	Father's
3	Mother's occupation: Agriculture	Mother's education	Region: l
4	Household wealth	Household wealth	Househol
5	Type of residence	Father's occupation: Agriculture	Region: l
6	Skilled birth attendance	Father's occupation: Other	Region: l
7	Region: Equateur	Region: Kinshasa	NA
8	Father's occupation: Household, unskilled, not working	Region: Equateur	NA
9	Region: Kinshasa	Region: Katanga	NA
10	Region: Sud-Kivu	Region: Bandundu	NA

9 Save Results Objects

The last chunk assembles the full object list and writes it to disk for the main report to consume.

```
drc_results_objects <- list(
  generation_params = runtime_params,
  source_data_file = source_data_file,
  paper_reference_file = paper_reference_file,
  outcome_labels = outcome_labels,
  congo_model_dt = congo_model_dt,
  raw_congo_model_dt = raw_congo_model_dt,
```

```

congo_var_labels = congo_var_labels,
raw_congo_var_labels = raw_congo_var_labels,
congo_predictors = congo_predictors,
raw_congo_predictors = raw_congo_predictors,
indicator_level_map = indicator_level_map,
congo_ci_formula = congo_ci_formula,
data_summary = drc_data_summary_table,
outcome_distribution = outcome_plot_dt,
candidate_predictors = candidate_predictor_table,
root_ci = drc_root_ci_table,
regression_model_summary = regression_model_summary,
classical_decomposition = classical_decomposition_table,
classical_decomposition_grouped = classical_decomposition_grouped,
published_drc_table3_reference = published_drc_table3_reference,
published_drc_table3_comparison = published_drc_table3_comparison,
rineq_summary = rineq_summary,
tree_tuning = tree_tuning,
tree_best_by_type = tree_best_by_type,
tree_best_controls = tree_best_controls,
tree_selection = tree_selection_table,
tree_fit_table = tree_fit_table,
tree_complexity_metrics = tree_complexity_metrics,
tree_min_relative_gain_path = tree_min_relative_gain_path,
tree_models_by_type = tree_models_by_type,
selected_tree_type = selected_tree_type,
selected_tree_fit = selected_tree_fit,
selected_tree_summary = selected_tree_summary,
all_tree_terminal_summaries = all_tree_terminal_summaries,
tree_variable_importance = tree_variable_importance,
lecturer_rpart_models = lecturer_rpart_models,
lecturer_rpart_summary = lecturer_rpart_summary,
forest_tuning = forest_tuning,
forest_best_by_type = forest_best_by_type,
forest_best_controls = forest_best_controls,
forest_fit_table = forest_fit_table,
forest_complexity_metrics = forest_complexity_metrics,
forest_min_relative_gain_path = forest_min_relative_gain_path,
selected_forest_fit = selected_forest_fit,
forest_surrogates_by_type = forest_surrogates_by_type,
selected_forest_surrogate = selected_forest_surrogate,
selected_forest_surrogate_summary = selected_forest_surrogate_summary,
forest_selection = forest_selection_table,
forest_variable_importance = forest_variable_importance,
predictive_forest_tuning = ranger_tuning,
predictive_forest_cv_predictions = ranger_cv_predictions,
predictive_forest_threshold_performance = ranger_threshold_performance,
selected_predictive_forest_threshold = selected_ranger_threshold,
predictive_forest_sampling = ranger_sampling_summary,
predictive_forest_feature_lookup = ranger_feature_lookup,
predictive_forest_fit = predictive_forest_fit,
shap_contributions = selected_shap_contributions,
shap_diagnostics = selected_shap_diagnostics,
shap_summary_long = shap_summary_long,
method_comparison = method_comparison_table,
importance_wide = importance_wide
)

save(drc_results_objects, file = results_object_file)

```

```
data.table(
  saved_to = results_object_file,
  object_count = length(drc_results_objects),
  object_names = paste(names(drc_results_objects), collapse = ", ")
)
```

```

                                                    saved_to
                                                    <char>
1: C:/Users/moses.mburu.FIND/Pictures/personal/thesis-mse/data/drc_report_results_objects.rda
  object_count
    <int>
1:          61

1: generation_params, source_data_file, paper_reference_file, outcome_labels, congo_model_dt, raw_con
```